CCF-CSP 前三题备考讲义

第一题：基础实现　第二题：工具与算法　第三题：综合模拟

2026 年 10 月 9 日

使用说明

本讲义面向已经学过变量、分支、循环、数组和函数，希望系统训练 CCF-CSP 认证前三题的学生。学习主线是：先把短程序写准确，再学习减少重复计算的方法，最后把长题意拆成可靠的程序模块。

正文中的教学例题均给出完整的任务、输入输出、规模限制、样例、解答和参考实现，可以脱离外部网站学习。例题是围绕原资料知识点重新组织的教学题，**不等同于所链接平台题目的完整原版题面**。原题链接用于进一步练习，不能直接把教学例题的代码提交到规则不同的原题。

每章建议按三个步骤学习：先自己推演例题，再解释参考代码中的关键不变量，最后遮住答案完成章末练习。不会做时优先寻找“状态究竟表示什么”“哪一步重复计算”“哪个边界没有定义”，而不是立即记忆模板。

参考程序以 C 风格为主：使用 scanf/printf、普通数组、下标循环、结构体和独立函数；需要动态容器时再使用 STL。没有使用 auto、匿名函数或结构化绑定。STL 属于 C++，因此统一按 C++17 编译，不能把包含 STL 的程序当成纯 C 源文件提交。程序逐项包含实际需要的标准头文件。全局数组会自动初始化为 0；局部普通数组不会，使用前必须初始化。数组容量按本讲义题目范围预留，迁移到别题时要重新检查。

全书分为三个大篇章。**第一篇**：**第一题**，学习第 1–2 章的基础输入输出、枚举、统计和小模拟，不依赖 STL。**第二篇**：**第二题**，先在第 3 章逐项学习 STL，再学习第 4–9 章的前缀和、二分、贪心、搜索、动态规划与背包。**第三篇**：**第三题**，第 10 章训练字符串解析，第 11 章展开五个完整模拟案例，第 12 章提供综合训练与复盘。提高专题放在书末附录，供完成主线后选读。

这种划分是一条学习路线，不表示每次考试的题号与算法固定对应。第一题通常优先用基础实现；第二篇里的容器也为第三题管理复杂状态服务。附录知识可按需要回查，无须先学完全部提高内容才开始前三题训练。

一个始终要回答的问题

一段代码通过样例，只说明它在那个输入上表现正确。每次写完都应检查：最小规模、最大规模、重复值、边界位置、不可达状态，以及题面允许但样例没有出现的情况。

目录

第一部分　第一题：基础实现　6

1　备考方法与程序基本功　6

1.1　从题面到程序的五个问题　6

1.2　复杂度决定能否完成计算　6

1.3　数值范围与输入边界　6

1.4　例题统计一组整数　7

1.5　练习与解答　8

2　基础枚举与小模拟　8

2.1　模拟的核心是状态时刻　8

2.2　例题网格的一轮同步更新　9

2.3　矩形相交的边界　9

2.4　日期模拟　10

2.5　练习与解答　10

第二部分　第二题：STL 与常用算法　11

3　STL 容器与常用算法详解　11

3.1　头文件、类型与最小程序　11

3.2　区间与迭代器：所有接口的共同基础　12

3.3　sort：普通数组排序与多关键字排序　12

3.4　例题按成绩和编号排序　13

3.5　unique：去相邻重复与修改逻辑长度　14

3.6　lower_bound 与 upper_bound：四种边界　15

3.7　离散化：值域大时保存相对大小　15

3.8　vector：可变长数组　16

3.9　string：与字符数组配合处理文本　17

3.10　pair：两个值的组合　19

3.11　stack：后进先出与括号匹配　19

3.12　queue：先进先出与处理顺序　20

3.13　priority_queue：堆与优先处理　21

3.14　set：自动去重的有序集合　23

3.15 multiset：重复值与删除一个副本 23
3.16 map：键到信息的映射 24
3.17 常用算法补充与容器选择 26
3.18 练习与解答 27

4 前缀和与差分 **27**
4.1 一维前缀和的推导 27
4.2 例题静态区间求和 27
4.3 二维前缀和与容斥 28
4.4 差分把区间修改变成端点修改 28
4.5 例题批量区间加法 29
4.6 二维差分的四个角 29
4.7 练习与解答 30

5 二分与双指针 **30**
5.1 二分依赖单调性 30
5.2 例题切木条的最大整数长度 30
5.3 跳石头中的判定建模 31
5.4 双指针如何减少重复枚举 31
5.5 例题乘积小于 k 的连续区间个数 31
5.6 差值数对与重复元素 32
5.7 练习与解答 32

6 贪心与基础数学 **33**
6.1 贪心需要证明 33
6.2 例题合并石堆的最小代价 33
6.3 中位数最小化绝对距离和 34
6.4 质因数分解与溢出检查 34
6.5 矩阵运算先看维度 35
6.6 练习与解答 35

7 搜索与剪枝 **35**
7.1 搜索状态和递归职责 35
7.2 例题按字典序输出全排列 35
7.3 剪枝必须有排除理由 36
7.4 例题迷宫最短步数 36

7.5 访问标记的三种含义 . . . 37
7.6 练习与解答 . . . 37

8 动态规划入门 38

8.1 先定义状态再写转移 . . . 38
8.2 例题最长严格上升子序列 . . . 38
8.3 例题非空最大子段和 . . . 39
8.4 例题数字三角形的最大路径和 . . . 39
8.5 二维 DP 与最长公共子序列 . . . 40
8.6 记忆化搜索 . . . 40
8.7 练习与解答 . . . 41

9 基础背包与计数 41

9.1 先确定问题问的是什么 . . . 41
9.2 例题每件物品最多选一次 . . . 41
9.3 例题每种物品可以无限选择 . . . 42
9.4 用最小反例检查循环方向 . . . 43
9.5 例题无限硬币的组合数 . . . 43
9.6 例题每组最多选一件 . . . 44
9.7 恰好装满时如何处理不可达 . . . 45
9.8 练习与解答 . . . 45

第三部分 第三题：字符串解析与综合模拟 46

10 字符串处理与解析基础 46

10.1 常用操作与区间含义 . . . 46
10.2 例题带引号的单行分词 . . . 46
10.3 C 字符数组与 string 的区别 . . . 47
10.4 解析与业务处理分开 . . . 47
10.5 练习与解答 . . . 47

11 第三题综合模拟专题 48

11.1 长题意的实现流程 . . . 48
11.2 例题路径规范化 . . . 48
11.3 例题授权和撤销的状态管理 . . . 49
11.4 例题带括号的整数表达式 . . . 51

11.5 例题树形文档的后代选择查询 52
11.6 例题模板展开与循环依赖 54
11.7 练习与解答 56

12 综合训练与复盘 **57**

12.1 练习方式 57
12.2 例题训练 A 差值数对 57
12.3 例题训练 B 矩形增量的最大值 58
12.4 例题训练 C 非抢占任务调度 58
12.5 如何复盘一次错误 60
12.6 提交前检查表 61

A 提高专题与模型扩展 **61**

A.1 多重背包与二进制拆分 61
A.2 混合背包与多维费用 61
A.3 方案恢复 62
A.4 计数背包的撤销 62
A.5 LIS 优化与排列 LCS 62
A.6 实数二分与三分的前提 63
A.7 可删堆的延迟删除 63
A.8 字符串匹配的线性预处理 63
A.9 附录练习与解答 64

B 资料来源与继续练习 **64**

第一部分　第一题：基础实现

本篇目标是把短题意翻译成准确的数组、循环和状态更新。先练输入输出与边界，再练枚举、统计和同步模拟。基础解法能清楚表达规则时，优先采用基础解法。

1　备考方法与程序基本功

1.1　从题面到程序的五个问题

读题后先写下：输入包含什么；输出要求什么；一个对象需要哪些属性；一次操作如何改变状态；数据规模允许多少工作量。例如，统计学生成绩时，对象可以是“编号与成绩”，状态是所有学生记录，输出可能是排序结果或某种统计值。不要把没有读懂的规则直接埋进条件分支。

遇到长题意，先处理名词和动词。名词通常对应结构体、数组或映射；动词通常对应函数。对“先……再……”“同时……”“若不存在……”等词逐一标记，它们常决定程序是否正确。

1.2　复杂度决定能否完成计算

设输入规模为 n。一次遍历是 $O(n)$，两层独立遍历通常是 $O(n^2)$；每次将区间减半的查找是 $O(\log n)$。排序通常需要 $O(n\log n)$ 比较。复杂度表示增长趋势，不是精确运行秒数。

若 $n = 2\times10^5$，两重枚举约需要 4×10^{10} 次配对，通常不可行；排序的比较数量级约为几百万，更值得考虑。若 $n \le 20$，枚举全部子集有 $2^{20} = 1048576$ 种，可能合适。实际限制还与常数、内存、实现和时限有关，不能把某个次数阈值当作保证。

操作	典型复杂度	常见改进方向
每次重新求区间和	$O(nq)$	静态数组用前缀和降为 $O(n+q)$。
枚举所有数对	$O(n^2)$	利用排序、映射或双指针。
每个状态重复递归	可能指数级	完整状态可缓存时使用记忆化。
每次扫描取最小值	每次 $O(n)$	动态维护可用优先队列。
每个答案都尝试	与答案范围成正比	判定具有单调性时考虑二分。

1.3　数值范围与输入边界

常见 32 位有符号整数的最大值约为 2.1×10^9。若 $n = 2\times10^5$ 且每项绝对值不超过 10^9，总和可能达到 2×10^{14}，需要 long long。尤其注意：两个 int 先相乘再赋给 long long，乘法仍可能先溢出。应写 `1LL*a*b`，让乘法从一开始就在更宽的类型中进行。

整数用 `scanf("%d",&n)`，long long 用`%lld`；输出格式要匹配类型。字符数组用带宽度限制的 `scanf("%100s",s)` 读入单词，数组至少有 101 格。fgets 读取整行并保留可能出现的换行。先读数

字再读整行时，应处理数字所在行剩余的字符；若空行或前导空格有意义，不要用跳过所有空白的方法。第 3 章的 string 小节给出完整示例。

统一区间含义也很重要。数组前缀和常采用 $[l, r]$ 闭区间；STL 的迭代器区间通常采用 $[first, last)$，last 指向最后一个元素之后。两种习惯都可以使用，但转换处必须明确。

1.4 例题统计一组整数

题目。输入 n 个整数，输出总和、最小值、最大值，以及出现次数最多的值及其次数。若多个值并列，输出最小的值。$1 \le n \le 2 \times 10^5$，$|a_i| \le 10^9$。

输入。第一行 n，第二行 n 个整数。**输出**。第一行依次输出总和、最小值、最大值；第二行输出众数及出现次数。

输入

```
6
3 -1 3 2 -1 5
```

输出

```
11 -1 5
-1 2
```

解答。总和与极值可边读边更新；最小值不能初始化为 0，否则全正数时会出错。众数作为本例的进阶要求：先调用 C 标准库 qsort 排序，再扫描每个相等值段。按值升序扫描且只在次数严格增加时更新答案，就保留了并列时较小的值。qsort 的比较函数返回负数、0 或正数，表示小于、等于或大于；不要用可能溢出的相减判断。扫描为 $O(n)$，空间为 $O(n)$，总时间由排序主导。C 标准未规定 qsort 的复杂度保证；需要明确的 $O(n \log n)$ 比较次数保证时，可使用第二篇讲解的 sort。初学时先掌握总和与极值，再学习众数部分。

完整参考实现 01_stats.cpp

```cpp
#include <cstdio>
#include <cstdlib>
#include <climits>
long long a[200005];
int compare(const void *p, const void *q) {
    long long x = *(const long long *)p;
    long long y = *(const long long *)q;
    if (x < y) return -1;
    if (x > y) return 1;
    return 0;
}
int main() {
    int n;
    scanf("%d", &n);
    long long sum = 0, mn = LLONG_MAX, mx = LLONG_MIN;
    for (int i = 0; i < n; i++) {
        scanf("%lld", &a[i]); sum += a[i];
        if (a[i] < mn) mn = a[i];
        if (a[i] > mx) mx = a[i];
    }
    qsort(a, n, sizeof(a[0]), compare);
    long long mode = a[0];
    int best = 0, i = 0;
    while (i < n) {
        int j = i;
        while (j < n && a[j] == a[i]) j++;
        if (j - i > best) { best = j - i; mode = a[i]; }
        i = j;
    }
    printf("%lld %lld %lld\n%lld %d\n", sum, mn, mx, mode, best);
    return 0;
}
```

1.5 练习与解答

练习 1。数组长度为 10^5，每项不超过 10^9，总和是否可以用 int？**解答**。最大总和为 10^{14}，应使用 long long。若单项也可能相乘，还要检查乘法是否提前提升类型。

练习 2。对 $[4,4,4]$，上题输出什么？**解答**。第一行是 `12 4 4`，第二行是 `4 3`。这检查重复值与极值更新。

练习 3。有 10^5 次查询，每次扫描 10^5 个数求和，工作量约是多少？**解答**。约 10^{10} 次元素访问。若数组不变，应预处理前缀和；若存在修改，则先判断能否离线处理，不直接套静态前缀和。

2 基础枚举与小模拟

2.1 模拟的核心是状态时刻

模拟题要求按规则执行，并不意味着随意堆条件。先定义变量表示“操作之前”还是“操作之后”，再明确更新顺序。若同一轮的所有变化依赖旧状态，应把旧状态和新状态分开。若操作依次发生，则后面的操作通常应看到前面操作的结果。

枚举题还需说明为什么不重不漏。例如枚举无序数对，使用 $1 \le i < j \le n$，每对只出现一次；若使用 $i \neq j$，则每对通常出现两次。判断条件正确和枚举范围正确同样重要。

2.2 例题网格的一轮同步更新

题目。给定一个 $n \times m$ 的 0/1 网格。一次更新后，每个格子的新值由其旧状态的上下左右四邻格决定：值为 1 的邻格至少有两个，则新值为 1，否则为 0。越界邻格不计入。只进行一轮更新，$1 \le n, m \le 500$。

输入。第一行 n, m，随后每行 m 个 0 或 1。**输出**。更新后的网格。

输入

```
2 3
1 0 1
0 1 0
```

输出

```
0 1 0
1 0 1
```

推演。左上格的两个有效邻格都是 0，所以变为 0；上方中间格的三个邻格都是 1，所以变为 1。所有计数都使用原数组 a，把答案放进 b。若直接覆盖 a，后面的格子会读到已经变化的数据。

正确性。程序对每格恰好检查四个方向，只有合法坐标参与计数，因此统计值与题意一致；写入 b 不改变 a，满足同步更新。时间和空间均为 $O(nm)$。

完整参考实现 02_grid.cpp

```cpp
#include <cstdio>
const int N = 505;
int a[N][N], b[N][N];
int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};
int main() {
    int n, m;
    scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) scanf("%d", &a[i][j]);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            int count = 0;
            for (int k = 0; k < 4; k++) {
                int x = i + dx[k], y = j + dy[k];
                if (x >= 0 && x < n && y >= 0 && y < m)
                    count += a[x][y];
            }
            b[i][j] = (count >= 2);
        }
    }
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            printf("%d%c", b[i][j], j + 1 == m ? '\n' : ' ');
    return 0;
}
```

2.3 矩形相交的边界

两个与坐标轴平行的矩形分别为 $[x_1, x_2] \times [y_1, y_2]$ 和 $[u_1, u_2] \times [v_1, v_2]$，且各端点已经有序。相交宽度与高度为

$$w = \max(0, \min(x_2, u_2) - \max(x_1, u_1)), \qquad h = \max(0, \min(y_2, v_2) - \max(y_1, v_1)).$$

面积为 wh。例如 $[0,3]\times[0,2]$ 与 $[2,5]\times[1,4]$ 的交集宽、高都为 1，面积为 1。仅接触边界时面积为 0。若题目统计整数格点而非连续面积，端点规则不同，不能擅自加 1 或减 1。

2.4 日期模拟

日期问题先确定历法。公历闰年的判定是：能被 400 整除，或者能被 4 整除但不能被 100 整除。月份长度用表保存，二月按闰年调整。把日期加一天时，先增加日；若超过当月天数，日改为 1 并增加月；若月变成 13，再进入下一年。每步只有一个明确定义的状态变化。

```
bool leap(int y) {
    return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
}
int days(int y, int m) {
    int d[] = {0,31,28,31,30,31,30,31,31,30,31,30,31};
    return d[m] + (m == 2 && leap(y));
}
```

2.5 练习与解答

练习 1。对只有一个格子的网格，原值为 1，更新后是多少？**解答**。为 0，因为没有有效邻格。它检查越界和最小规模。

练习 2。2000 年和 2100 年是否闰年？**解答**。2000 能被 400 整除，是闰年；2100 能被 100 整除但不能被 400 整除，不是闰年。因此两年的二月分别有 29 天和 28 天。

练习 3。矩形 $[0,2]\times[0,2]$ 与 $[2,4]\times[0,2]$ 的相交面积是多少？**解答**。宽度为 0，面积为 0；边接触不产生正面积。

练习 4。若网格更新两轮，是否可以一直只计算 b？**解答**。不可以。第一轮结束后应交换 a、b，使第二轮从第一轮结果出发；每轮仍必须完整覆盖 b 的全部位置。

第二部分　第二题：STL 与常用算法

本篇先建立完整的 STL 工具箱，再学习如何减少重复计算。容器操作、复杂度和算法适用条件要一起理解；本篇的 string、map 和 set 也会直接服务于第三篇。

3　STL 容器与常用算法详解

本章是第二篇的工具基础。STL 通常指标准模板库；本讲义将常用的标准库容器、算法、字符串和数对放在一起讲解。它们仍然属于 C++，程序需要用 C++ 编译器。**采用 C 风格写法**是指优先使用 scanf/printf、普通数组、下标循环、结构体和独立函数；使用 STL 时保留必要的模板类型和成员函数调用。

学习时先掌握 sort、二分查找、vector 与基本队列，再学 set、map 和堆。string、嵌套容器与自定义比较器会在第三篇继续用到。每种工具都要回答：数据如何存放、能做什么操作、一次操作花多少时间、什么情况下不能调用。

3.1　头文件、类型与最小程序

C++ 可以调用 C 标准库。`<cstdio>` 提供 scanf/printf，`<cstring>` 提供 strlen/strcmp，`<algorithm>` 提供 sort、unique 等算法。容器各有头文件；只包含 algorithm 并不能保证 queue 可用。

头文件	工具	适合解决的问题
algorithm	sort、unique、二分、reverse	排序、去重、静态有序查找。
vector	vector	元素数变化，需要按下标访问。
string	string	保存、比较和加工文本。
utility	pair、make_pair	把两个有关联的值放在一起。
stack	stack	后进先出，撤销、括号匹配。
queue	queue、priority_queue	先来先处理，或优先处理极值。
set	set、multiset	动态有序集合，前驱、后继。
map	map	从名字、编号映射到信息或次数。
functional	greater	常用于声明小根堆。

本章短代码默认已经包含对应头文件，且写在 main 或指定函数中；给出的完整参考程序可以直接编译。`using namespace std;` 允许省略 std:: 前缀。`vector<int>` 表示元素为 int 的动态数组，尖括号里的类型不是容量。

```cpp
#include <cstdio>
#include <vector>
using namespace std;
int main() {
    vector<int> a;
    a.push_back(7);
    printf("%d\n", a[0]);
    return 0;
}
```

读法。a 是容器；点号后是它的成员操作；push_back 在末尾添加一个元素。普通数组也可以配合 STL 算法，不必把所有数组改成 vector。

3.2 区间与迭代器：所有接口的共同基础

3.2.1 左闭右开区间

算法通常接收 $[first, last)$：包含 first，不包含 last。数组前 n 项是 `[a,a+n)`；使用 1 下标时则是 `[a+1,a+n+1)`。空区间满足 first 等于 last，长度为 0。

```cpp
int a[5] = {9, 4, 7, 1, 3};
sort(a + 1, a + 4); // Sort indices 1, 2, 3 only.
// a is now {9, 1, 4, 7, 3}.
```

3.2.2 迭代器可以移动，但不一定能相减

`begin()` 指向首元素，`end()` 指向最后一个元素之后。`*it` 读取当前元素，`++it` 移动到下一个。空容器的 begin 等于 end；end 不能解引用。

```cpp
vector<int> a(3, 8);
vector<int>::iterator it;
for (it = a.begin(); it != a.end(); ++it)
    printf("%d\n", *it);
```

vector 迭代器支持 `it+3` 和相减；set/map 迭代器只支持逐步前后移动，不能写 `it+3`、`s[0]` 或用迭代器相减求排名。迭代器也不能跨两个不同容器比较或相减。

3.2.3 什么时候需要重新取得迭代器

vector 扩容会使原来的指针、引用和迭代器失效；即使没有扩容，在中间插入、删除后，受影响位置及后面的迭代器也不能继续照用。set/map 插入通常不影响已有迭代器，删除则使指向被删元素的迭代器失效。最稳妥的初学者习惯是：修改容器后重新查找，删除时使用 erase 返回的下一个位置。

3.3 sort：普通数组排序与多关键字排序

3.3.1 升序、降序和局部排序

默认 sort 按小于关系升序排列。下例里的 cmp 是**普通函数**，不是匿名函数；返回 true 表示 x 应排在 y 前面。

```cpp
bool descending(int x, int y) { return x > y; }
// Inside main:
int a[5] = {3, 1, 4, 1, 5};
sort(a, a + 5);              // 1 1 3 4 5
sort(a, a + 5, descending); // 5 4 3 1 1
```

对 n 个元素，sort 保证 $O(n \log n)$ 量级的比较次数。不要把它固定理解为某一种朴素快速排序实现。sort 不保证等价元素的原相对顺序；需要保留时用 stable_sort，或将原下标加入次关键字。

3.3.2 比较器必须严格

`cmp(x,x)` 必须为 false；x 排在 y 前时，y 不能也排在 x 前；比较关系还要传递。`x>=y` 不是合格比较器。不能在比较过程中随机改变规则，也不应通过 `return x-y;` 返回布尔结果：非零都会变成 true，而且减法可能溢出。字符数组按内容比较应使用 strcmp，不能直接比较两个数组地址。

3.4 例题按成绩和编号排序

题目。输入 n 名学生的编号和成绩，成绩降序，相同成绩按编号字典序升序。编号是不含空格的字符串，长度 1 到 30 且互不相同；$1 \le n \le 2 \times 10^5$，成绩 0 到 100。首行 n，随后每行编号和成绩，输出排序后的记录。

输入

```
4
b02 90
a01 95
a02 90
a03 90
```

输出

```
a01 95
a02 90
a03 90
b02 90
```

解答。结构体数组保存记录，cmp 先判断成绩，再用 strcmp 判断编号。编号数组容量 31，多留一格保存字符串结束符。`%30s` 限制一次最多读入 30 个字符。比较次数为 $O(n \log n)$，每次字符串比较最多扫描 30 个字符，记录存储为 $O(n)$。

完整参考实现 03_rank.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <algorithm>
using namespace std;
const int N = 200005;
struct Student { char id[31]; int score; };
Student a[N];
bool cmp(Student x, Student y) {
    if (x.score != y.score) return x.score > y.score;
    return strcmp(x.id, y.id) < 0;
}
int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%30s%d", a[i].id, &a[i].score);
    sort(a, a + n, cmp);
    for (int i = 0; i < n; i++) printf("%s %d\n", a[i].id, a[i].score);
    return 0;
}
```

自测。若编号为 a2 和 a10，字典序中 a10 在 a2 之前；题目若要求按数字编号排序，应先把数字部分解析为数值。

3.5 unique：去相邻重复与修改逻辑长度

unique 把相邻等价元素压缩到区间前部，返回**新的逻辑末尾**，不改变普通数组容量，也不会自动缩短 vector。新末尾后的元素不属于去重结果，不要依赖其具体值。

```cpp
int a[6] = {3, 1, 3, 2, 2, 1};
sort(a, a + 6);                 // 1 1 2 2 3 3
int m = unique(a, a + 6) - a;   // m = 3
for (int i = 0; i < m; i++) printf("%d ", a[i]);
// Output: 1 2 3
```

如果不排序，`[1,2,1]` 中两个 1 不相邻，unique 不会删掉后面的 1。若要求保留首次出现顺序，排序去重会改变顺序，应该用集合判断是否见过，同时另存答案。vector 全局去重的常见顺序是 sort、unique、erase：

```cpp
vector<int> v;
v.push_back(3); v.push_back(1); v.push_back(3);
sort(v.begin(), v.end());
vector<int>::iterator last = unique(v.begin(), v.end());
v.erase(last, v.end());
```

练习与完整解答：有序去重。输入 n 个整数，$1 \le n \le 2 \times 10^5$、$|a_i| \le 10^9$。第一行输出不同值数量，第二行按升序输出这些值。输入 `6` 和 `3 1 3 2 2 1`，输出 `3` 和 `1 2 3`。排序后每个等值段保留一个即可，时间 $O(n \log n)$。

完整参考实现 27_distinct.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
int a[200005];
int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    sort(a, a + n);
    int m = unique(a, a + n) - a;
    printf("%d\n", m);
    for (int i = 0; i < m; i++)
        printf("%d%c", a[i], i + 1 == m ? '\n' : ' ');
    return 0;
}
```

3.6 lower_bound 与 upper_bound：四种边界

本节默认序列按升序排列；不要在无序数组上直接使用二分查找。lower_bound 找第一个不小于 x 的位置，upper_bound 找第一个大于 x 的位置；找不到时都返回右端点。

```cpp
int a[4] = {2, 2, 5, 7};
int n = 4, x = 2;
int l = lower_bound(a, a + n, x) - a;
int r = upper_bound(a, a + n, x) - a;
printf("%d %d %d\n", l, r, r - l); // 0 2 2
if (l < n && a[l] == x) puts("FOUND");
```

需求	下标公式	必须检查
第一个 $\geq x$	lower_bound 减起点	下标小于 n。
第一个 $> x$	upper_bound 减起点	下标小于 n。
最后一个 $< x$	lower_bound 下标减 1	结果不小于 0。
最后一个 $\leq x$	upper_bound 下标减 1	结果不小于 0。
x 出现次数	upper 下标减 lower 下标	两次使用同一有序区间。

数组或 vector 上的一次查询为 $O(\log n)$。例如 x=8 时两端均为 4，表示不存在 8；`a[4]` 不是有效元素。**set/map 上优先使用成员函数**，如 `s.lower_bound(x)`；通用算法作用于树迭代器时，比较次数虽少，迭代器移动却可能达到线性数量。

3.7 离散化：值域大时保存相对大小

离散化把实际值变成在去重有序表中的排名。原序列 $[100, 5, 100, 20]$ 的有序不同值为 $[5, 20, 100]$，1 起始排名为 $[3, 1, 3, 2]$。**保留相等关系与大小关系，不保留坐标距离。**

```cpp
int a[4] = {100, 5, 100, 20}, keys[4], rank_id[4];
int n = 4;
for (int i = 0; i < n; i++) keys[i] = a[i];
sort(keys, keys + n);
int m = unique(keys, keys + n) - keys;
for (int i = 0; i < n; i++)
    rank_id[i] = lower_bound(keys, keys + m, a[i]) - keys + 1;
```

排序 $O(n \log n)$，逐项查询 $O(n \log n)$，空间 $O(n)$。必须复制一份数据再排序，否则原顺序被破坏。涉及线段长度或面积时，用排名定位，用 keys 恢复原坐标计算距离。在线出现的新值不能随意追加到末尾并假定排名仍正确。

3.8 vector：可变长数组

3.8.1 建立、赋值与下标访问

```cpp
vector<int> a;              // Empty, size 0.
vector<int> b(5);           // Five zeros.
vector<int> c(5, -1);       // Five copies of -1.
a.push_back(10); a.push_back(20);
printf("%d %d\n", (int)a.size(), a[1]); // 2 20
b = a;                     // Copies the elements.
for (int i = 0; i < (int)b.size(); i++) printf("%d ", b[i]);
```

合法下标为 0 到 size 减 1。`a[5]=1` 不会自动扩容。size 返回无符号类型；本讲义数据量可用 int 表示，循环前转为 int，避免空容器上 `size()-1` 发生无符号下溢。`front()`、`back()`、`pop_back()` 均应先确认非空。

3.8.2 size、capacity、resize、reserve 的区别

size 是已有元素数，capacity 是无需重新分配时能容纳的数量。resize 改变元素数；reserve 只预留容量。`vector<int> a; a.reserve(100);` 之后 a 仍然为空，不能写 a[0]，应该 push_back 或先 resize。`a.resize(100,7)` 只把新增元素填成 7，原有元素不会全部改写；要整体重置可用 `a.assign(100,7)`。

3.8.3 插入、删除和二维 vector

```cpp
vector<int> a(3, 5);        // 5 5 5
a.insert(a.begin() + 1, 9); // 5 9 5 5
a.erase(a.begin() + 2);     // 5 9 5
a.erase(a.begin(), a.begin() + 2); // 5
a.clear();                 // size becomes 0.
vector<vector<int> > grid(3, vector<int>(4, 0));
grid[1][2] = 8;
```

下标访问为 $O(1)$，末尾 push_back 均摊 $O(1)$；单次扩容可能搬移全部元素。中间插入删除通常为 $O(n)$，连续在开头删除 n 次可能变成 $O(n^2)$。clear 清空元素，不承诺立刻归还所有容量。二维 vector 的每一行是独立 vector，不能假定跨行内存连续；规模已知的普通矩阵通常直接使用二维数组。

练习与解答：追加和撤销。首行 q，$1 \le q \le 10^5$；随后每条命令为 ADD x、UNDO 或 PRINT，$|x| \le 10^9$。ADD 末尾追加，UNDO 删除末尾一项，空时忽略；PRINT 输出当前序列，空时输出

EMPTY。保证全部 PRINT 实际输出的元素数不超过 2×10^5。

输入

```
7
UNDO
ADD 4
ADD 9
PRINT
UNDO
PRINT
UNDO
```

输出

```
4 9
4
```

每次修改仅涉及末尾，vector 适合保存操作结果。时间为 $O(q+S)$ 的均摊量级，S 为输出元素总数；没有对 S 的约束时，反复 PRINT 的输出本身也可能很大。

完整参考实现 33_undo.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <vector>
using namespace std;
int main() {
    int q;
    scanf("%d", &q);
    vector<int> a;
    while (q--) {
        char op[10]; scanf("%9s", op);
        if (strcmp(op, "ADD") == 0) {
            int x; scanf("%d", &x); a.push_back(x);
        } else if (strcmp(op, "UNDO") == 0) {
            if (!a.empty()) a.pop_back();
        } else {
            if (a.empty()) puts("EMPTY");
            else {
                for (int i = 0; i < (int)a.size(); i++)
                    printf("%d%c", a[i], i + 1 == (int)a.size() ? '\n' : ' ');
            }
        }
    }
    return 0;
}
```

3.9 string：与字符数组配合处理文本

3.9.1 读写、复制和字典序

string 自动管理存储空间，可赋值、拼接和按内容比较。字符数组用 `strcmp(a,b)`，string 可以直接用 `a==b`、`a<b`。字符串`"10"` 按字典序小于`"2"`，这不等于数值比较。char 指针不应直接作为按文本内容比较的 map 键，本章统一使用 string 键。

```
char buf[101];
scanf("%100s", buf);
string s = buf;
printf("%s\n", s.c_str());
string t = s; t += "!";
for (int i = 0; i < (int)t.size(); i++) printf("%c", t[i]);
```

不能写 `scanf("%s", s)` 或 `printf("%s", s)`：格式化函数不会自动把 string 转换成字符指针。输出使用 c_str；输入先读字符数组再赋给 string。`c_str()` 返回的指针只宜用于即时读取，修改 string 后不要继续依赖旧指针。

3.9.2 整行输入与空格

```
char line[1005];
if (fgets(line, sizeof(line), stdin) != NULL) {
    line[strcspn(line, "\r\n")] = '\0';
    string s = line;
    printf("[%s]\n", s.c_str());
}
```

fgets 最多读入容量减 1 个字符，可能保留换行。例中用 strcspn 找到第一个回车或换行并截断。容量必须覆盖题目允许的最大行长，否则一次 fgets 不一定读完一行。先 scanf 读取数字再 fgets 时，应把该行余下字符读到换行；只调用一次 getchar 仅在确知剩余恰好一个换行时才足够。不要用跳过所有空白的方法处理允许空行或前导空格的题目。

3.9.3 截取、查找、插入、删除与替换

```
string s = "abcdef";
string t = s.substr(2, 3);  // "cde": start, count.
s.erase(1, 2);              // "adef": remove b,c.
s.insert(1, "XY");          // "aXYdef".
s.replace(1, 2, "-");       // "a-def".
string::size_type p = s.find("de");
if (p != string::npos) printf("%d\n", (int)p); // 2
```

substr 第二参数是长度，不是终点。find 失败返回 string::npos，先判断成功再转成 int 或拿来访问。insert/erase/replace 可能移动后缀，反复从头修改长字符串会很慢；可以逐段追加到新字符串。string 不能用 memset 清零，清空用 clear。

练习与解答：**从左向右替换**。输入三行 s、p、t，各行长度不超过 1000，允许空格；s 和 t 可空，p 非空。按原串 s 从左到右查找 p，每次选取最靠左的不重叠出现，将其替换为 t。新插入的内容不再次参与匹配，输出结果。例如 s 为 aaaa、p 为 aa、t 为 b，结果 bb；s 为 aaa 时结果 ba。

输入

```
one two one
one
X
```

输出

```
X two X
```

用 start 标记原串尚未处理的位置。找到后先追加未匹配片段，再追加 t，随后前进 p 的长度。始终在原串上 find，可避免 t 包含 p 时重复替换甚至死循环。答案最长为约 10^6 字符；朴素匹配可用

$O(|s||p| + |answer|)$ 估计，具体 string 查找实现可能优化，本题规模不依赖这种优化。

完整参考实现 `34_replace.cpp`

```cpp
#include <cstdio>
#include <cstring>
#include <string>
using namespace std;
char line[1005], pattern[1005], replacement[1005];
void read_line(char s[]) {
    if (fgets(s, 1005, stdin) == NULL) s[0] = '\0';
    s[strcspn(s, "\r\n")] = '\0';
}
int main() {
    read_line(line); read_line(pattern); read_line(replacement);
    string s = line, p = pattern, t = replacement, answer;
    string::size_type start = 0;
    while (true) {
        string::size_type pos = s.find(p, start);
        if (pos == string::npos) break;
        answer += s.substr(start, pos - start);
        answer += t;
        start = pos + p.size();
    }
    answer += s.substr(start);
    printf("%s\n", answer.c_str());
    return 0;
}
```

3.10 pair：两个值的组合

pair 是辅助类型，不是可增删元素的容器。`pair<int,int>` 可以存坐标、数值与下标等；成员名为 first、second。为了保持简单写法，本章用 make_pair 创建数对，不用结构化绑定。

```cpp
pair<int, int> p = make_pair(3, 8);
printf("%d %d\n", p.first, p.second);
pair<int, int> a[3];
a[0] = make_pair(2, 1);
a[1] = make_pair(1, 9);
a[2] = make_pair(1, 4);
sort(a, a + 3); // (1,4), (1,9), (2,1)
```

默认按 first 比较，first 相同才比较 second，即字典序。字段有明确业务含义时可改用结构体并命名为 x、y、score 等，提高可读性；pair 的比较不会自动满足“第一项降序、第二项升序”，这种规则仍需自定义比较函数。

3.11 stack：后进先出与括号匹配

3.11.1 基本操作

```cpp
stack<int> st;
st.push(4); st.push(9);
if (!st.empty()) {
    printf("%d\n", st.top()); // 9
    st.pop();
}
printf("%d\n", (int)st.size()); // 1
```

push 加入栈顶，top 只读取栈顶，pop 只删除栈顶，**pop 不返回被删除的值**。默认底层容器下

这些操作为常数时间。stack 不提供下标访问或普通遍历；需要访问所有元素时可改用 vector。普通数组也能实现栈：用 top 表示已有元素数，入栈 `a[top++]=x`，读栈顶 `a[top-1]`，出栈 `top--`，访问前确认 top 大于 0。

练习与解答：三类括号匹配。输入一行仅由 `()[]{}` 组成的字符串，长度 0 到 10^5；空串合法。若每个右括号能与最近尚未匹配的同类左括号配对，输出 YES，否则 NO。输入 `([]){}` 输出 YES，输入 `([)]` 输出 NO。

最近出现的未匹配左括号必须先配对，所以使用栈。扫描右括号时先检查非空，再检查类型；扫描结束还必须检查栈是否为空。每个字符最多入栈出栈各一次，时间和最坏空间均为 $O(n)$。只比较左右括号总数会误判顺序或嵌套类型。

完整参考实现 28_brackets.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <stack>
using namespace std;
char s[100005];
int match(char left, char right) {
    return (left == '(' && right == ')') ||
           (left == '[' && right == ']') ||
           (left == '{' && right == '}');
}
int main() {
    if (fgets(s, sizeof(s), stdin) == NULL) s[0] = '\0';
    s[strcspn(s, "\r\n")] = '\0';
    stack<char> st;
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '(' || c == '[' || c == '{') st.push(c);
        else {
            if (st.empty() || !match(st.top(), c)) {
                puts("NO"); return 0;
            }
            st.pop();
        }
    }
    puts(st.empty() ? "YES" : "NO");
    return 0;
}
```

3.12 queue：先进先出与处理顺序

```cpp
queue<int> q;
q.push(4); q.push(9);
if (!q.empty()) {
    printf("%d %d\n", q.front(), q.back()); // 4 9
    q.pop();
}
```

push 在队尾添加，front 读取队首，back 读取队尾，pop 删除队首；访问空队列是错误。默认底层容器下这些操作为常数时间。queue 按进入顺序处理，不按数值大小排序。BFS 按层扩展用 queue；总要取最大值时则应考虑 priority_queue。

普通数组实现队列时，可用 $[head, tail)$ 表示有效区间：入队 `a[tail++]=x`，出队先读 `a[head]` 再 head 加 1，非空条件 head 小于 tail。只增不回绕的写法需要容量覆盖**累计入队次数**；长期复用时可改成循环队列或直接用 queue。

练习与解答：**排队取号**。首行 n，$1 \le n \le 10^5$；随后每行 PUSH x 或 POP，编号 x 为 int 范围内整数。PUSH 入队；POP 输出并删除队首，空队列输出 EMPTY。

输入
6
POP
PUSH 5
PUSH 2
POP
POP
POP
输出
EMPTY
5
2
EMPTY

维护进入顺序即可。每条命令 $O(1)$，总时间 $O(n)$，最多保存 n 个编号。

完整参考实现 29_queue.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <queue>
using namespace std;
int main() {
    int n;
    scanf("%d", &n);
    queue<int> q;
    while (n--) {
        char op[10];
        scanf("%9s", op);
        if (strcmp(op, "PUSH") == 0) {
            int x; scanf("%d", &x); q.push(x);
        } else {
            if (q.empty()) puts("EMPTY");
            else { printf("%d\n", q.front()); q.pop(); }
        }
    }
    return 0;
}
```

3.13 priority_queue：堆与优先处理

3.13.1 大根堆与小根堆

```cpp
priority_queue<int> large;
large.push(4); large.push(9); large.push(2);
printf("%d\n", large.top()); // 9
priority_queue<int, vector<int>, greater<int> > small;
small.push(4); small.push(9); small.push(2);
printf("%d\n", small.top()); // 2
```

第一个模板参数是元素类型，第二个是底层存储，第三个是比较方式。默认大根堆 top 最大；greater 使最小值位于堆顶。top 为 $O(1)$，push/pop 的堆调整为 $O(\log n)$，底层 vector 分配另有均

摊成本。堆不是完全排好序的数组，不支持第 k 项查询，也没有 erase(x) 接口。要删除指定值可用 multiset；延迟删除见提高附录。

3.13.2 结构体与普通比较函数

堆的比较函数返回 true，表示**左边对象优先级比右边低**，与 sort 里“应放在前面”的直观含义不同。下面优先取成绩更高者，相同成绩取编号更小者。函数指针类型写法中，Compare 表示接收两个 Item 并返回 bool 的函数指针。

```cpp
struct Item { int score, id; };
bool lower_priority(Item a, Item b) {
    if (a.score != b.score) return a.score < b.score;
    return a.id > b.id;
}
// Inside main:
typedef bool (*Compare)(Item, Item);
priority_queue<Item, vector<Item>, Compare> q(lower_priority);
Item a; a.score = 90; a.id = 2; q.push(a);
a.id = 1; q.push(a);
printf("%d\n", q.top().id); // 1
```

构造时必须传入比较函数，不能让函数指针保持空值。不要直接把 sort 的“高分在前”比较器原封不动放进堆。本讲义第三篇的任务调度继续使用这种普通函数写法。

练习与解答：**保留最大的 k 个数**。输入 n、k 及 n 个整数，$1 \le k \le n \le 2 \times 10^5$，数值在 int 范围内。重复值按不同出现计数，按降序输出最大的 k 个数。样例：`5 3` 和 `4 1 9 9 2`，输出 `9 9 4`。

维护最多 k 个元素的小根堆。每读入一个数就入堆，超过 k 个时删除最小值；留下的恰好是已读前缀中最大的 k 个。反复弹出得到升序，最后逆转。时间 $O(n \log(k+1) + k)$，额外空间 $O(k)$。若 n 很小且一次读完，直接 sort 也是合理选择。

完整参考实现 `32_heap.cpp`

```cpp
#include <cstdio>
#include <queue>
#include <vector>
#include <algorithm>
#include <functional>
using namespace std;
int main() {
    int n, k;
    scanf("%d%d", &n, &k);
    priority_queue<int, vector<int>, greater<int> > q;
    for (int i = 0; i < n; i++) {
        int x; scanf("%d", &x); q.push(x);
        if ((int)q.size() > k) q.pop();
    }
    vector<int> a;
    while (!q.empty()) { a.push_back(q.top()); q.pop(); }
    reverse(a.begin(), a.end());
    for (int i = 0; i < (int)a.size(); i++)
        printf("%d%c", a[i], i + 1 == (int)a.size() ? '\n' : ' ');
    return 0;
}
```

3.14 set：自动去重的有序集合

set 存唯一键，默认按升序排列。insert 重复键不会新增元素；find 失败返回 end；count 在 set 中只能为 0 或 1。查找、插入、按键删除的通常复杂度为 $O(\log n)$，完整遍历为 $O(n)$。

```
set<int> s;
s.insert(5); s.insert(2); s.insert(5);
printf("%d\n", (int)s.size()); // 2
set<int>::iterator it;
for (it = s.begin(); it != s.end(); ++it) printf("%d ", *it);
if (s.find(5) != s.end()) puts("FOUND");
s.erase(2);
```

不能通过 `*it=` 新值修改 set 中的键，否则会破坏有序关系；修改键应先删旧值再插新值。遍历不是按插入顺序，不支持按下标读取。只做一次排序去重时普通数组往往更直接；数据边加入边查询时 set 更合适。

3.14.1 前驱、后继与安全边界

以下寻找严格小于 x 的前驱，以及不小于 x 的后继。两者定义不同于“严格大于”的后继；严格大于应改用 upper_bound。

```
int x = 6;
set<int>::iterator it = s.lower_bound(x);
if (it != s.end()) printf("next = %d\n", *it);
if (it != s.begin()) {
    --it;
    printf("previous = %d\n", *it);
}
```

空集合 begin 等于 end，两种访问都会被挡住；x 大于所有元素时 it 为 end，可以在集合非空的条件下先减一次取得最大值；x 不大于最小值时不能对 begin 做减法。

3.15 multiset：重复值与删除一个副本

multiset 允许相同值出现多次。`erase(x)` 删除**所有等于 x 的元素**；`erase(it)` 只删除迭代器指向的一个。count 可能返回大于 1，统计大量重复元素的成本也与出现次数有关，不能把它一律算作 $O(\log n)$。

```
multiset<int> s;
s.insert(5); s.insert(5); s.insert(8);
multiset<int>::iterator it = s.find(5);
if (it != s.end()) s.erase(it); // One 5 remains.
printf("%d\n", (int)s.count(5)); // 1
s.erase(5); // Remove all remaining 5s.
```

练习与解答：动态候选值。首行 q，$1 \le q \le 2 \times 10^5$；随后每行 ADD x、DEL x 或 GE x，$|x| \le 10^9$。ADD 加入一个副本，DEL 删除一个副本，不存在则忽略；GE 输出当前最小的、不小于 x 的值，不存在输出 NONE。

输入

```
8
ADD 5
ADD 5
ADD 9
DEL 5
GE 5
DEL 5
GE 5
GE 10
```

输出

```
5
9
NONE
```

使用 multiset 保留重复值；DEL 采用 find 加 erase 迭代器；GE 用成员 lower_bound。每条操作至多 $O(\log(q+1))$，空间 $O(q)$。

完整参考实现 30_neighbors.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <set>
using namespace std;
int main() {
    int q;
    scanf("%d", &q);
    multiset<int> s;
    while (q--) {
        char op[10]; int x;
        scanf("%9s%d", op, &x);
        if (strcmp(op, "ADD") == 0) s.insert(x);
        else if (strcmp(op, "DEL") == 0) {
            multiset<int>::iterator it = s.find(x);
            if (it != s.end()) s.erase(it);
        } else {
            multiset<int>::iterator it = s.lower_bound(x);
            if (it == s.end()) puts("NONE");
            else printf("%d\n", *it);
        }
    }
    return 0;
}
```

3.16 map：键到信息的映射

3.16.1 下标、find 和有序遍历

map 保存唯一键及其对应值，默认按键排序。迭代器的 first 是键、second 是值；键不可直接修改，值可以修改。

例如，string 到 int 的映射可以保存名字及其次数；int 到 vector 的映射可以保存某个编号对应的一组记录。

```cpp
map<string, int> cnt;
cnt["apple"]++; cnt["pear"] += 2;
map<string, int>::iterator it = cnt.find("orange");
if (it == cnt.end()) puts("NOT FOUND");
for (it = cnt.begin(); it != cnt.end(); ++it)
    printf("%s %d\n", it->first.c_str(), it->second);
```

访问不存在键的 `cnt[key]` 会插入默认值；这里 int 默认为 0，因此计数很方便。但只想查存在性时应使用 find 或 count，避免创建新键。`cnt.count(key)` 表示这个键是否存在，不是该单词的频数。`insert(make_pair(key,value))` 遇到已有键不会覆盖旧值；需要覆盖应使用下标赋值。

比较和树查找通常需要 $O(\log n)$ 次键比较。若键是长度 L 的字符串，一次比较最坏 $O(L)$，整体可估为 $O(L \log n)$。值域很小且连续的计数优先用普通数组，避免不必要的树节点开销。

练习与解答：**单词计数**。输入 n 及 n 个仅含小写字母的单词，每词长度 1 到 30，$1 \le n \le 10^5$。按字典序输出每个不同单词及次数。

输入

```
5
pear apple pear banana apple
```

输出

```
apple 2
banana 1
pear 2
```

读入时对键计数；map 已经按键排列，最后顺序遍历即可，不需要再 sort。若有 m 个不同单词，计数约需 $O(30n \log(m+1))$ 时间，存储约 $O(30m)$。

完整参考实现 `31_frequency.cpp`

```cpp
#include <cstdio>
#include <map>
#include <string>
using namespace std;
int main() {
    int n;
    scanf("%d", &n);
    map<string, int> count;
    for (int i = 0; i < n; i++) {
        char word[31];
        scanf("%30s", word); count[word]++;
    }
    map<string, int>::iterator it;
    for (it = count.begin(); it != count.end(); ++it)
        printf("%s %d\n", it->first.c_str(), it->second);
    return 0;
}
```

3.16.2 遍历时删除与嵌套映射

```cpp
map<string, int>::iterator it = cnt.begin();
while (it != cnt.end()) {
    if (it->second == 0) it = cnt.erase(it);
    else ++it;
}
map<string, set<string> > rights;
rights["alice"].insert("file1");
map<string, set<string> >::iterator p = rights.find("alice");
if (p != rights.end() && p->second.count("file1")) puts("YES");
```

erase 之后继续对旧 it 加 1 会使用失效迭代器。嵌套 map/set 适合权限管理：外层找用户，内层查资源。查询用 find，授权才用下标主动创建，这正是第三篇授权模拟的状态组织方式。

3.17 常用算法补充与容器选择

```cpp
int a[5] = {1, 2, 3, 4, 5};
reverse(a, a + 5); // 5 4 3 2 1
fill(a, a + 3, 7); // 7 7 7 2 1
int pos = max_element(a, a + 5) - a;
printf("%d %d\n", pos, a[pos]); // 0 7
swap(a[0], a[4]);
```

reverse 和 fill 为线性，max_element 找最大值位置也为线性；空区间会返回右端点，不能解引用。min/max 常用于两个值的比较；数组整体求最小值用 min_element。给普通 int 数组全部置 0 可以 memset；**memset 按字节填充**，`memset(a,1,sizeof(a))` 不是把每项变成整数 1。给每项赋一般数值时用 fill 或 for 循环。

任务特征	首选工具	选择依据
规模已知、频繁下标访问	普通数组	简单直接，容量需提前留够。
末尾追加、撤销，偶尔遍历	vector	连续存储，动态长度。
最近未完成对象先处理	stack	后进先出。
最早进入对象先处理	queue	先进先出。
每次仅取最大或最小	priority_queue	堆顶快，不提供任意删除。
动态查前驱后继、无重复	set	有序唯一键。
动态查前驱后继、有重复	multiset	保留出现次数。
按名字查次数或属性	map	键到值，有序遍历。
静态排序后重复查找	数组加二分	不需维护动态树结构。

unordered_map、unordered_set 不按键排序，常见哈希操作平均较快但最坏可能退化；不能替代需要 lower_bound 或有序输出的 map/set。deque 支持两端操作，list 适合特定位置已知的链表操作，但本讲义常规题型不以它们为主。选择容器应由题目操作决定，不能仅凭“高级”程度选择。

3.18 练习与解答

1. 有序数组 $[2,2,5,7]$ 中，x=2 的 lower 和 upper 下标是多少？x=8 呢？**解答**：分别为 0、2 和 4、4；后者是末尾，不能访问数组第 4 项。

2.vector 只 reserve(20)，能否写 a[0]=3？**解答**：不能，size 仍为 0。push_back(3) 或先 resize 才建立元素。

3.map 中 a 键的频数是 5，`cnt.count("a")` 是多少？**解答**：1；它统计键是否存在。实际频数在 it->second 或 cnt[”a”] 中。

4.multiset 含 $[4,4,8]$，删一个 4 后如何保证仍有一个 4？**解答**：find(4)，检查不为 end，然后 erase 该迭代器；不能直接 erase(4)。

5. 比较器返回 `x.score>=y.score` 是否正确？**解答**：错误，自比较会为 true，破坏严格弱序。相同主关键字再比较次关键字。

6. 栈和队列均依次加入 2、7、3，再依次弹出，输出是什么？**解答**：栈为 3、7、2，队列为 2、7、3；默认大根堆则为 7、3、2。

7. 把坐标 $[1,100,101]$ 变成 $[1,2,3]$ 后能否用排名差算距离？**解答**：不能。排名只保序，原前两点距离 99 而排名差为 1。

8. 替换题 s=aaa、p=aa、t=aaaa 时，结果和处理次数是多少？**解答**：一次匹配，输出 aaaaa；新插入内容不再匹配，避免无休止增长。

9. 用小根堆保存最大的 3 个数，已读 $[6,1,5,2]$ 时堆中是什么？**解答**：保留 2、5、6，堆顶 2；内部数组次序不必完全升序。

10. 为什么不能在 map 查询缺失用户时一律写 rights[user]？**解答**：该语句会建立空集合；查询越多，状态可能无故增长。使用 find 后再访问已有记录。

接口核对。进一步可查阅 C++ 工作草案中的vector 容量规则、关联容器规则、优先队列与排序算法。上述定义、示例和解答均已写在正文，链接仅用于查阅细节。

4 前缀和与差分

4.1 一维前缀和的推导

定义 $pre[0]=0$，$pre[i]=a_1+\cdots+a_i$。则

$$pre[i]=pre[i-1]+a_i,\qquad \sum_{i=l}^{r}a_i=pre[r]-pre[l-1].$$

原因是前一个总和包含 1 到 r，减去 1 到 $l-1$ 后只剩所需区间。预处理 $O(n)$，每次查询 $O(1)$。该方法适合**原数组不变**的多次查询。

4.2 例题静态区间求和

输入长度为 n 的数组和 q 次闭区间查询，每次输出区间和。$1\le n,q\le 2\times 10^5$，$|a_i|\le 10^9$，保证 $1\le l\le r\le n$。第一行 n,q，第二行数组，随后 q 行各给出 l,r。

输入

```
5 3
2 -1 3 4 5
1 3
2 4
5 5
```

输出

```
4
6
5
```

前缀数组为 $[0,2,1,4,8,13]$，查询 $[2,4]$ 得 $8-2=6$。0 号位置让 $l=1$ 的查询无需特殊判断。使用 long long 存前缀和。总复杂度 $O(n+q)$，空间 $O(n)$。

完整参考实现 05_prefix.cpp

```cpp
#include <cstdio>
const int N = 200005;
long long pre[N];
int main() {
    int n, q;
    scanf("%d%d", &n, &q);
    for (int i = 1; i <= n; i++) {
        long long x;
        scanf("%lld", &x);
        pre[i] = pre[i - 1] + x;
    }
    while (q--) {
        int l, r;
        scanf("%d%d", &l, &r);
        printf("%lld\n", pre[r] - pre[l - 1]);
    }
    return 0;
}
```

4.3 二维前缀和与容斥

设 $S[i][j]$ 为从 $(1,1)$ 到 (i,j) 的矩形和，0 行和 0 列为 0：

$$S[i][j] = S[i-1][j] + S[i][j-1] - S[i-1][j-1] + a[i][j].$$

左上角被前两项算了两次，因此减一次。查询左上 (x_1,y_1)、右下 (x_2,y_2) 的矩形：

$$S[x_2][y_2] - S[x_1-1][y_2] - S[x_2][y_1-1] + S[x_1-1][y_1-1].$$

例如矩阵 $\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$ 的第 2–3 列总和是 $2+3+5+6=16$；用公式得到 $21-0-5+0=16$。预处理 $O(nm)$，每个查询 $O(1)$。

4.4 差分把区间修改变成端点修改

定义 $d[1]=a[1]$，$d[i]=a[i]-a[i-1]$。原数组是差分数组的前缀和。给 $[l,r]$ 每个元素加 v，只需

$$d[l] \mathrel{+}= v, \qquad d[r+1] \mathrel{-}= v.$$

前一个修改让从 l 开始的前缀和都增加 v，后一个修改从 r+1 起抵消。必须为 n+1 位置预留空间。该做法适合执行全部修改后再求最终结果。

4.5 例题批量区间加法

输入长度为 n 的数组，执行 q 次操作，每次给闭区间 $[l,r]$ 加 v，最后输出整个数组。$1 \le n,q \le 2\times 10^5$，初始值和修改值绝对值不超过 10^9。

输入

```
5 2
1 2 3 4 5
2 4 3
1 5 -1
```

输出

```
0 4 5 6 4
```

先用初始数组构造差分。所有修改都可叠加，最后做一次前缀累加。总时间 $O(n+q)$，空间 $O(n)$。上例第一轮得到 $[1,5,6,7,5]$，第二轮整体减 1。

完整参考实现 `06_difference.cpp`

```cpp
#include <cstdio>
const int N = 200005;
long long a[N], d[N];
int main() {
    int n, q;
    scanf("%d%d", &n, &q);
    for (int i = 1; i <= n; i++) {
        scanf("%lld", &a[i]);
        d[i] = a[i] - a[i - 1];
    }
    while (q--) {
        int l, r; long long v;
        scanf("%d%d%lld", &l, &r, &v);
        d[l] += v; d[r + 1] -= v;
    }
    long long x = 0;
    for (int i = 1; i <= n; i++) {
        x += d[i];
        printf("%lld%c", x, i == n ? '\n' : ' ');
    }
    return 0;
}
```

4.6 二维差分的四个角

给矩形 $[x_1,x_2]\times[y_1,y_2]$ 加 v：

```
d[x1][y1] += v;
d[x2+1][y1] -= v;
d[x1][y2+1] -= v;
d[x2+1][y2+1] += v;
```

随后对 d 求二维前缀和，就得到每个格子的增量。左上角启动影响，下方和右方分别取消，右下角因取消两次再补一次。若有初始矩阵，可先恢复增量再加到初值上，或按 $d[i][j]=a[i][j]-a[i-1][j]-a[i][j-1]+a[i-1][j-1]$ 建初始差分。

4.7 练习与解答

练习 1。数组 $[3,-2,7]$ 的前缀和与差分是什么？**解答**。前缀和含 0 号为 $[0,3,1,8]$；差分为 $[3,-5,9]$。注意差分可以为负。

练习 2。在全 0 的 3×3 网格中，给左上 2×2 加 2，再给右下 2×2 加 3，结果是什么？**解答**。

$$\begin{pmatrix}2&2&0\\2&5&3\\0&3&3\end{pmatrix}.$$

中心格被覆盖两次，得到 5。按四角更新再恢复，应得到同样矩阵。

练习 3。修改和查询交替出现，是否仍能用上述普通差分每次 $O(1)$ 回答区间和？**解答**。不能。普通差分方便累计修改，但恢复全部值需要一次扫描。需要重新分析是否可离线，或使用树状数组等动态结构，不能隐瞒恢复成本。

5 二分与双指针

5.1 二分依赖单调性

二分的对象可以是有序数组，也可以是答案。核心是判定结果沿某个方向只变化一次。例如长度 x 能切出至少 k 段木头，那么更短的正整数长度也能做到；可行区间是前缀。先写出“真、真、假、假”的顺序，再选择搜索第一个真或最后一个真。

在闭区间 $[lo,hi]$ 搜索最后一个可行值时，取上中位数 $mid=lo+(hi-lo+1)/2$。若 mid 可行，令 lo=mid；否则 hi=mid-1。上中位数保证剩下两个元素时仍能缩小区间。每步至少排除一个候选，区间最终只剩答案。

5.2 例题切木条的最大整数长度

有 n 根木条，第 i 根长度为 a_i。每根可切成若干长度相同的整段，剩余部分丢弃，不能拼接不同木条。至少要得到 k 段，求最大正整数段长；若连长度 1 都无法满足，输出 0。$1\le n\le 2\times10^5$，$1\le a_i\le10^9$，$1\le k\le10^{12}$。输入第一行 n、k，第二行各长度。

```
输入
4 7
20 15 10 17
输出
7
```

长度 7 时可得 $2+2+1+2=7$ 段；长度 8 时只有 $2+1+1+2=6$ 段。判定函数为 $\sum_i\lfloor a_i/x\rfloor\ge k$，随 x 增大计数不增。0 作为“无正整数解”的哨兵，不传入判定函数，因此不会除以 0。复杂度 $O(n\log\max a_i)$，空间 $O(n)$。

完整参考实现 07_wood.cpp

```cpp
#include <cstdio>
const int N = 200005;
int n;
long long k, a[N];
int check(long long len) {
    long long count = 0;
    for (int i = 0; i < n; i++) {
        count += a[i] / len;
        if (count >= k) return 1;
    }
    return 0;
}
int main() {
    scanf("%d%lld", &n, &k);
    long long lo = 0, hi = 0;
    for (int i = 0; i < n; i++) {
        scanf("%lld", &a[i]);
        if (a[i] > hi) hi = a[i];
    }
    while (lo < hi) {
        long long mid = lo + (hi - lo + 1) / 2;
        if (check(mid)) lo = mid;
        else hi = mid - 1;
    }
    printf("%lld\n", lo);
    return 0;
}
```

5.3 跳石头中的判定建模

若目标是让相邻保留石头的最小距离尽量大，可以二分距离 x。判定时从起点向后扫描，若当前石头与上次保留位置距离小于 x，则至少需要移走其中一块。通常贪心优先移走当前中间石头，以保留更靠前的落点，为后续留下空间。计数的含义应是**达到距离要求至少需移走的数量**，不是最多能移走多少。

终点不可移走，需要单独验证末段，或者采用已证明等价的计数处理。不要只在中间石头上检查距离。二分答案的困难往往在判定函数的正确性，而不是二分循环本身。

5.4 双指针如何减少重复枚举

设右端点逐渐增加，若满足条件的最小左端点不会向左移动，就可保存它而不重复扫描。两个指针分别最多移动 n 次，总移动次数为 $O(n)$。有两层循环不代表一定是 $O(n^2)$，要统计指针在全程移动多少次。

5.5 例题乘积小于 k 的连续区间个数

给定 n 个**正整数**，统计乘积严格小于 k 的非空连续子数组数量。$1 \le n \le 2 \times 10^5$，$1 \le a_i \le 10^6$，$0 \le k \le 10^9$。输入第一行 n、k，第二行数组。

输入

```
4 100
10 5 2 6
```

输出

```
8
```

合法区间包括 4 个单元素、$[10,5]$、$[5,2]$、$[2,6]$、$[5,2,6]$，共 8 个。维护窗口乘积，每加入右端点，就在乘积过大时删除左端点。缩小后，以当前右端点结尾的合法区间有 $right - left + 1$ 个。

所有数至少为 1，因此缩短区间不会使乘积增大，合法左端点形成后缀。若允许负数，这个性质会失效；若允许 0，除法维护也需另外处理。k 不大于 1 时没有合法非空区间。按本题约束，乘入新元素前乘积小于 10^9，乘后小于 10^{15}，long long 足够。时间 $O(n)$，空间 $O(n)$。

完整参考实现 08_window.cpp

```cpp
#include <cstdio>
const int N = 200005;
long long a[N];
int main() {
    int n; long long k;
    scanf("%d%lld", &n, &k);
    for (int i = 0; i < n; i++) scanf("%lld", &a[i]);
    if (k <= 1) { puts("0"); return 0; }
    long long product = 1, answer = 0;
    int left = 0;
    for (int right = 0; right < n; right++) {
        product *= a[right];
        while (left <= right && product >= k) product /= a[left++];
        answer += right - left + 1;
    }
    printf("%lld\n", answer);
    return 0;
}
```

5.6 差值数对与重复元素

给定正整数差 c，若已排序，可对每个值 x 查找 x+c 的出现次数。使用 upper_bound 与 lower_bound 之差可自然处理重复。若 x 有 p 个，x+c 有 q 个，贡献是 pq，而不是 1。例如 $[1,1,3,3,3]$ 且 c=2，答案是 $2 \times 3 = 6$。

当 c=0 时需先确认题目问有序数对还是无序数对，是否允许同一位置使用两次。无序且不同位置的同值组贡献为 $p(p-1)/2$，不能沿用 c>0 的计数公式。

5.7 练习与解答

练习 1。木条 $[1,1]$，k=3，答案是什么？**解答**。0。最大段长哨兵必须覆盖无解情况。

练习 2。窗口题数组 $[1,1,1]$，k=2，答案是多少？**解答**。全部 6 个非空连续区间都合法，分别贡献 1、2、3。k=1 时答案为 0。

练习 3。闭区间最后可行值搜索中，lo=2、hi=3，若使用下中位数且判定为真会怎样？**解答**。mid=2，更新 lo=2 后区间不变，可能死循环。应取上中位数，或采用另一套能证明缩小区间的写法。

6 贪心与基础数学

6.1 贪心需要证明

贪心是在当前状态下确定一个选择并且不回退。它成立需要证明：至少存在一个最优解包含这次选择。常用方法是交换论证，即把某个最优方案换成包含该选择的方案，而目标值不变差。找到几个正确样例不是证明。

例如按价值最大的物品优先装背包并不总是正确：容量 4，物品的（体积，价值）分别为 $(3,5),(2,4),(2,4)$，先拿价值 5 的物品只能得到 5，拿后两件可得 8。基础 01 背包通常需要 DP，不能仅因“选大的”直观就使用贪心。

6.2 例题合并石堆的最小代价

有 n 堆石子，每次可以任意选两堆合并，代价等于两堆石子数之和。求合成一堆的最小总代价。本题**没有相邻限制**。$1 \le n \le 2 \times 10^5$，每堆石子数在 1 到 10^9 之间。输入第一行 n，第二行各堆大小。

输入
```
4
1 2 3 4
```
输出
```
19
```

先合并 1 和 2，代价 3；再合并 3 和 3，代价 6；最后合并 4 和 6，代价 10。总计 19。

为什么取最小两堆。合并过程可以看成二叉树，每个原始堆是叶子；总代价等于每个叶子权值乘以其深度之和。在固定树形中，较小权值放到较深位置不会使代价增大。选择一对最深的兄弟叶子，可通过交换把最小的两个权值放到那里。因此存在一个最优方案先合并最小两堆；把它们缩成一堆后，剩下同类子问题。重复此选择即可。

用小根堆维护所有堆，每次取两个最小值再插入它们的和。时间 $O(n \log n)$，空间 $O(n)$。只有一堆时无需合并，答案为 0。

完整参考实现 09_merge.cpp

```cpp
#include <cstdio>
#include <queue>
#include <vector>
#include <functional>
using namespace std;
int main() {
    int n;
    scanf("%d", &n);
    priority_queue<long long, vector<long long>, greater<long long> > q;
    for (int i = 0; i < n; i++) {
        long long x;
        scanf("%lld", &x); q.push(x);
    }
    long long answer = 0;
    while (q.size() > 1) {
        long long x = q.top(); q.pop();
        long long y = q.top(); q.pop();
        answer += x + y; q.push(x + y);
    }
    printf("%lld\n", answer);
    return 0;
}
```

6.3 中位数最小化绝对距离和

若在数轴上选位置 x，使 $\sum_i |a_i - x|$ 最小，排序后取中位数即可。把最左和最右配对：对任何位于两者之间的 x，两项距离和都是两端距离；离开该区间只会增大。对称配对后，所有区间的公共部分包含中位数。偶数个元素时，中间两数之间的任意位置都可达到最小值。

例如 $[1, 2, 9]$ 取 x=2，距离和为 8；取平均数 4，距离和为 10。平均数与中位数解决的目标不同。

6.4 质因数分解与溢出检查

对正整数 x，从 2 开始试除。若 d 整除 x，反复除去并统计指数。循环条件可写 `d <= x/d`，避免计算 d^2 溢出。循环结束后若 x>1，它是最后一个质因子。1 没有质因子。

```cpp
// prime[] and exponent[] each need at least 64 elements.
int factorize(long long x, long long prime[], int exponent[]) {
    int total = 0;
    for (long long d = 2; d <= x / d; d++) {
        if (x % d != 0) continue;
        int count = 0;
        do { x /= d; count++; } while (x % d == 0);
        prime[total] = d; exponent[total++] = count;
    }
    if (x > 1) { prime[total] = x; exponent[total++] = 1; }
    return total;
}
```

最坏试除次数为 $O(\sqrt{x})$。如果要分解很多较大的数，应重新估计总复杂度，而不是只看一次分解。

6.5 矩阵运算先看维度

若 A 是 $n\times k$，B 是 $k\times m$，则 $C=AB$ 是 $n\times m$，有 $C_{ij}=\sum_{t=1}^{k}A_{it}B_{tj}$，朴素复杂度 $O(nkm)$。矩阵乘法一般不可交换。若题意出现转置或连乘，应先写出各中间矩阵的维度与计算量，再决定结合顺序。计算量可以因结合顺序变化而显著不同，但结果在精确算术下符合结合律。

6.6 练习与解答

练习 1。三堆 $[2,2,8]$ 的最优代价是多少？**解答**。先合并 2 和 2 得到 4，代价 4；再与 8 合并代价 12，总计 16。若先合并 2 和 8 则为 22。

练习 2。分解 360。**解答**。$360=2^3\cdot 3^2\cdot 5$。反复整除才能记录完整指数。

练习 3。A 为 10×100，B 为 100×5，C 为 5×50，先算哪一对更省？**解答**。$(AB)C$ 的乘加规模为 $10\cdot 100\cdot 5+10\cdot 5\cdot 50=7500$；$A(BC)$ 为 $100\cdot 5\cdot 50+10\cdot 100\cdot 50=75000$，前者更省。

7 搜索与剪枝

7.1 搜索状态和递归职责

写递归前先明确：一次调用负责哪一步、已经作出哪些选择、接下来有哪些合法选择、什么时候记录答案。回溯是撤销本次选择，让下一条分支从同一个父状态开始。常见错误是撤销不完整，导致不同分支互相污染。

7.2 例题按字典序输出全排列

输入 $1\le n\le 8$，输出 1 到 n 的所有排列，每行一个，按字典序升序。

输入

```
3
```

输出

```
1 2 3
1 3 2
2 1 3
2 3 1
3 1 2
3 2 1
```

path[0..depth-1] 保存已选的前缀，used[x] 标记 x 是否已在前缀中。每层从小到大尝试尚未使用的数，输出顺序自然为字典序。每个排列对应唯一一条长度 n 的路径，因此不重不漏。输出本身有 $n\cdot n!$ 个数字量级，时间 $O(n\cdot n!)$，递归与标记空间 $O(n)$。

完整参考实现 10_permutations.cpp

```cpp
#include <cstdio>
int n, path[10], used[10];
void dfs(int depth) {
    if (depth == n) {
        for (int i = 0; i < n; i++)
            printf("%d%c", path[i], i + 1 == n ? '\n' : ' ');
        return;
    }
    for (int x = 1; x <= n; x++) {
        if (used[x]) continue;
        used[x] = 1; path[depth] = x;
        dfs(depth + 1);
        used[x] = 0;
    }
}
int main() {
    scanf("%d", &n);
    dfs(0);
    return 0;
}
```

7.3 剪枝必须有排除理由

剪枝是提前排除不可能产生合法答案或不可能改善答案的分支。可行性剪枝例如皇后问题中已占用的列与对角线；最优性剪枝例如当前代价已不小于已知最优值，且所有后续代价非负。若后续允许负代价，后一判断就可能失效。

n 皇后按行放置时，记录列 c、对角线 $r+c$、另一方向 $r-c+n$ 是否已使用。每行只放一个皇后，所以行冲突由递归结构自动排除。剪枝减少搜索量，但不意味着最坏复杂度必然变成多项式。

7.4 例题迷宫最短步数

输入 $n \times m$ 字符网格，点号为可走格，井号为障碍。每步上下左右移动一格，求左上角到右下角的最少步数；不可达输出-1。$1 \le n, m \le 500$，起终点也可能是障碍。第一行 n、m，随后 n 行长度为 m 的字符串。

输入

```
3 4
....
.##.
....
```

输出

```
5
```

解答。所有移动代价相等，使用 BFS。起点距离为 0，邻居距离为当前距离加 1；入队时就记录距离，确保同一格最多入队一次。队列按距离层次扩展，因此首次到达就是最短距离。注意这里的“步数”不包含起点，单格可走迷宫答案为 0。时间与空间均为 $O(nm)$。

完整参考实现 11_bfs.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
const int N = 505;
char g[N][N];
int d[N][N], qx[N * N], qy[N * N];
int dx[4] = {-1, 1, 0, 0}, dy[4] = {0, 0, -1, 1};
int main() {
    int n, m;
    scanf("%d%d", &n, &m);
    for (int i = 0; i < n; i++) {
        scanf("%504s", g[i]);
        fill(d[i], d[i] + m, -1);
    }
    int head = 0, tail = 0;
    if (g[0][0] == '.') {
        d[0][0] = 0; qx[tail] = 0; qy[tail++] = 0;
    }
    while (head < tail) {
        int x = qx[head], y = qy[head]; head++;
        for (int k = 0; k < 4; k++) {
            int a = x + dx[k], b = y + dy[k];
            if (a < 0 || a >= n || b < 0 || b >= m) continue;
            if (g[a][b] == '#' || d[a][b] != -1) continue;
            d[a][b] = d[x][y] + 1;
            qx[tail] = a; qy[tail++] = b;
        }
    }
    printf("%d\n", d[n - 1][m - 1]);
    return 0;
}
```

7.5 访问标记的三种含义

全排列中的 used 是当前路径内是否使用，返回时必须清除。普通 BFS 的 visited 表示已首次到达，通常全程不清除。记忆化的标记表示这个**完整状态的答案已算出**，应与具体到达路径无关。把这三种标记混用，会造成漏解、死循环或重复计算。

若走到同一位置时“钥匙是否拿到”影响后续，则状态必须是（位置，钥匙状态）。只用位置作为 visited 会错误合并不同状态。若边权不同，普通 BFS 不再保证最小代价；应根据权值性质考虑 0–1 BFS 或最短路算法，不直接套网格模板。

7.6 练习与解答

练习 1。数组版全排列返回后，需要清空 path[depth] 吗？**解答**。不必。有效前缀长度由 depth 给出，下次选择会覆盖这一格，只有 depth 等于 n 时才输出。used[x] 必须恢复为 0，否则后续分支会错误地禁止使用 x。若用可变长 vector 保存路径，则应配对 push_back 与 pop_back，不能把两种实现的恢复规则混为一谈。

练习 2。BFS 为何在入队时标记，而不是出队时？**解答**。同一格可能被多个同层前驱发现，若出队才标记会重复入队。入队时记录距离既防重，也固定首次最短距离。

练习 3。只有两个格子，边代价分别可能为 1 或 100，按经过格子数最少就一定费用最小吗？**解答**。不一定。BFS 最小化边数，只有每条边的代价相等时才等价于最小费用。选择算法前必须看目

标函数。

8 动态规划入门

8.1 先定义状态再写转移

动态规划把问题拆成可重复利用的子问题。每道题都要写出五项：状态含义、转移关系、初始值、计算顺序、答案位置。只记转移公式，容易在初值和边界处出错。

所谓无后效性，是状态已经包含决定后续所需的信息。给定状态后，不必再区分到达它的不同历史。这不表示当前选择不会影响未来，当前选择当然会改变后续状态。若过去某项信息会改变可选操作，就应把它加进状态。

例如爬楼梯，每次走 1、2 或 3 阶，令 $f[i]$ 为从 0 阶走到 i 阶的方案数。按最后一步分类，有 $f[i] = f[i-1] + f[i-2] + f[i-3]$。令 $f[0] = 1$，负下标状态为 0。这里的 1 表示“什么也不走”的一种空方案。按 i 从小到大计算，最终答案是 f[n]。

8.2 例题最长严格上升子序列

输入 n 个整数,选择一些位置并保持原有顺序,使选出的值严格递增,求最大长度。$1 \le n \le 5000$，值的绝对值不超过 10^9。子序列可以不连续。第一行 n，第二行数组。

输入

```
6
3 1 2 2 5 4
```

输出

```
3
```

可取 $[1,2,5]$ 或 $[1,2,4]$。设 f[i] 为**以第 i 个元素结尾**的最长长度，

$$f[i] = 1 + \max\{f[j] : j < i,\ a_j < a_i\},$$

没有合法 j 时 f[i]=1。任何以 i 结尾的较长方案，倒数第二个位置必然是一个合法 j；反过来，把 a[i] 接到该 j 的方案后面也合法。因此枚举所有前驱就覆盖全部可能。答案是所有 f[i] 的最大值，不一定是 f[n]。时间 $O(n^2)$，空间 $O(n)$。

完整参考实现 `12_lis.cpp`

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
const int N = 5005;
int a[N], f[N];
int main() {
    int n, answer = 0;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    for (int i = 0; i < n; i++) {
        f[i] = 1;
        for (int j = 0; j < i; j++)
            if (a[j] < a[i]) f[i] = max(f[i], f[j] + 1);
        answer = max(answer, f[i]);
    }
    printf("%d\n", answer);
    return 0;
}
```

8.3 例题非空最大子段和

输入 n 个整数，求一个**连续且非空**区间的最大和。$1 \le n \le 2 \times 10^5$，$|a_i| \le 10^9$。

输入

```
5
-2 3 -1 4 -5
```

输出

```
6
```

设 ending 为以当前位置结尾的非空区间最大和。若使用当前数 x，则最优区间要么从 x 重新开始，要么延长前一位置的最优区间：$ending = \max(x, ending + x)$。全局答案记录所有位置的最大 ending。由于要求非空，初始化为第一个元素，不能初始化为 0。时间 $O(n)$，额外空间 $O(1)$。

完整参考实现 `13_maxsum.cpp`

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
int main() {
    int n; long long x;
    scanf("%d%lld", &n, &x);
    long long ending = x, answer = x;
    for (int i = 1; i < n; i++) {
        scanf("%lld", &x);
        ending = max(x, ending + x);
        answer = max(answer, ending);
    }
    printf("%lld\n", answer);
    return 0;
}
```

8.4 例题数字三角形的最大路径和

输入 n 行数字，第 i 行有 i 个数。从顶点出发，每次走到下一行的正下方或右下方，求到最后一行的最大路径和。$1 \le n \le 1000$，每个数的绝对值不超过 10^6。第一行 n，随后各行给出三角形。

输入

```
3
2
-1 3
4 -2 1
```

输出

```
6
```

二维状态 f[i][j] 表示到第 i 行第 j 个位置的最优值，合法前驱只有 $(i-1,j-1)$ 与 $(i-1,j)$。初值为顶点，非法位置为负无穷。转移

$$f[i][j] = a[i][j] + \max(f[i-1][j-1], f[i-1][j]).$$

上例路径 $2,3,1$ 得 6。因为每行只依赖前一行，可压缩成一维。**从右向左更新**才能确保 f[j-1] 尚未被本行覆盖。代码设置 f[1]=0 作为顶点前的虚拟起点，其他位置为负无穷。时间 $O(n^2)$，额外空间 $O(n)$。

完整参考实现 14_triangle.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
const int N = 1005;
const long long NEG = -(1LL << 60);
long long f[N], row[N];
int main() {
    int n;
    scanf("%d", &n);
    fill(f, f + n + 1, NEG); f[1] = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) scanf("%lld", &row[j]);
        for (int j = i; j >= 1; j--)
            f[j] = max(f[j], f[j - 1]) + row[j];
    }
    printf("%lld\n", *max_element(f + 1, f + n + 1));
    return 0;
}
```

8.5 二维 DP 与最长公共子序列

给两个长度分别为 n、m 的序列，令 $f[i][j]$ 为两个前缀的最长公共子序列长度。若最后两个元素相等，候选为 $f[i-1][j-1]+1$；还可舍弃任意一边末元素。统一写为

$$f[i][j] = \max(f[i-1][j], f[i][j-1],\ [a_i = b_j] \text{ 时的} f[i-1][j-1]+1).$$

空前缀长度为 0，按行列递增计算。时间 $O(nm)$，滚动数组可把空间压缩到 $O(m)$，但仍不减少时间。若 n、m 都到 10^5，仅压缩空间仍不能通过，需要利用题目的额外性质。

8.6 记忆化搜索

记忆化按需求递归计算状态，求完后缓存。它与按顺序递推的主要区别是访问顺序。缓存键必须包含完整状态；若结果还取决于一个没有放进键里的参数，缓存会错误复用。

不要简单地以“结果为 0”判断是否计算过，因为 0 可能是合法答案。可以单设 done 数组。一般 DP 要求依赖关系可以按某种顺序求值；有环图上的最短路不能靠“加个缓存”就自动解决。第 11 章模板展开会同时使用“正在计算”和“已完成”两种标记识别环。

8.7 练习与解答

练习 1。数组 $[5,4,3]$ 的严格 LIS 长度是多少？**解答**。1。每个位置的 f 都为 1；若题目要求不下降序列，应把比较条件改为不大于，而不是仍用严格小于。

练习 2。数组 $[-5,-2,-7]$ 的非空最大子段和是多少？**解答**。-2。初始化为 0 会错误地选择空区间。

练习 3。三阶楼梯、每步走 1、2 或 3 阶，共多少方案？**解答**。4 种：111、12、21、3。由 $f[0]=1, f[1]=1, f[2]=2$ 得 $f[3]=4$。

练习 4。滚动数组能否把 LCS 的 $O(nm)$ 时间降为 $O(m)$？**解答**。不能。它只减少保存的历史行数，仍需要计算 n 行各 m 个状态。

9 基础背包与计数

9.1 先确定问题问的是什么

背包的“容量”可以是时间、预算或其他资源，“价值”可以是收益、数量或得分。建模前必须区分：资源是最多使用还是恰好使用；每件只能选一次还是可无限选择；目标是最大价值、最小代价、能否达到还是方案数量。

状态含义	初始化	典型答案
容量不超过 v 的最大价值	全 0，允许什么都不选	f[V]。
恰好用 v 的最大价值	f[0]=0，其余负无穷	f[V]；不可达需另行输出。
恰好用 v 的最小代价	f[0]=0，其余正无穷	f[V]。
恰好凑出 v 的方案数	f[0]=1，其余 0	f[V]，必要时取模。

9.2 例题每件物品最多选一次

有 n 件物品，每件体积 c、价值 w，容量为 V，求总体积不超过 V 的最大总价值。$1 \le n \le 200$，$0 \le V \le 5000$，$1 \le c \le 5000$，$0 \le w \le 10^9$。输入第一行 n、V，后面每行 c、w。

输入
3 5
2 3
3 4
4 5
输出
7

令 f[i][v] 为只考虑前 i 件物品、容量不超过 v 的最大价值。第 i 件选或不选：

$$f[i][v] = \max(f[i-1][v],\ f[i-1][v-c_i] + w_i),$$

第二项只在 $v \geq c_i$ 时存在。若压缩掉 i 维，容量必须从大到小更新，使 f[v-c] 仍是上一层状态，避免同一件被重复选。复杂度 $O(nV)$，空间 $O(V)$。

完整参考实现 15_knapsack01.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
long long f[5005];
int main() {
    int n, capacity;
    scanf("%d%d", &n, &capacity);
    for (int i = 0; i < n; i++) {
        int cost; long long value;
        scanf("%d%lld", &cost, &value);
        for (int v = capacity; v >= cost; v--)
            f[v] = max(f[v], f[v - cost] + value);
    }
    printf("%lld\n", f[capacity]);
    return 0;
}
```

9.3 例题每种物品可以无限选择

输入格式与上一题相同，但现在 n 表示物品种类，每种可以选任意非负整数件。仍要求体积不超过 V、最大化价值，且体积必须为正。采用同样的约束。

输入
2 7
2 3
3 4
输出
10

选择体积 2 的两件和体积 3 的一件，价值 10。二维转移的选取项变为 $f[i][v-c_i]+w_i$，它允许当前种类继续使用。因此一维容量循环从小到大，使 f[v-c] 已经包含本种物品。时间 $O(nV)$，空间 $O(V)$。

完整参考实现 16_knapsack_complete.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
long long f[5005];
int main() {
    int n, capacity;
    scanf("%d%d", &n, &capacity);
    for (int i = 0; i < n; i++) {
        int cost; long long value;
        scanf("%d%lld", &cost, &value);
        for (int v = cost; v <= capacity; v++)
            f[v] = max(f[v], f[v - cost] + value);
    }
    printf("%lld\n", f[capacity]);
    return 0;
}
```

9.4 用最小反例检查循环方向

只有一件体积 2、价值 3 的物品，容量 4。01 背包答案为 3；若正序更新，计算 f[4] 时使用了本轮刚更新的 f[2]=3，得到 6，实际上重复选择了同一件。完全背包允许重复，答案才是 6。记住“逆序 01、正序完全”之外，更要能解释读到的是哪一层。

9.5 例题无限硬币的组合数

有 n 种**互不相同的正整数面额**，每种无限个，问恰好凑出 target 的组合数，硬币选择顺序不算不同方案。输出对 10^9+7 取模的结果。$1 \le n \le 100$，$0 \le target \le 10^4$，面额不超过 10^4。输入第一行 n、target，第二行所有面额。

输入

```
3 5
1 2 5
```

输出

```
4
```

四种为 $1+1+1+1+1$、$1+1+1+2$、$1+2+2$、5。令 f[v] 为已处理面额能凑出的组合数。先枚举面额，再正序枚举金额，确保每种组合按固定面额顺序生成一次。f[0]=1 是构造非空方案的起点。时间 $O(n \cdot target)$，空间 $O(target)$。

完整参考实现 17_coin_count.cpp

```cpp
#include <cstdio>
const int MOD = 1000000007;
int f[10005];
int main() {
    int n, target;
    scanf("%d%d", &n, &target);
    f[0] = 1;
    for (int i = 0; i < n; i++) {
        int c;
        scanf("%d", &c);
        for (int v = c; v <= target; v++) f[v] = (f[v] + f[v - c]) % MOD;
    }
    printf("%d\n", f[target]);
    return 0;
}
```

若改成先枚举金额、再枚举最后一个硬币，就会按排列计数。例如面额 1 和 2 凑 3，组合只有 $1+1+1$ 与 $1+2$ 两种，排列则还区分 $1+2$ 和 $2+1$，共三种。循环顺序改变了状态的计数含义。

9.6 例题每组最多选一件

有 g 组物品，每组最多选一件，也可以不选。求容量 V 内的最大价值。$1 \le g \le 100, 0 \le V \le 2000$，每组 1 到 20 件，体积为正、价值非负。输入第一行 g、V，随后每组先给物品数 k，再给 k 对体积、价值。

输入

```
2 5
2
2 3
3 5
2
2 4
4 7
```

输出

```
9
```

第一组选体积 3 价值 5，第二组选体积 2 价值 4。对每组保留旧数组 f，所有选择都从旧状态转移到 next_f，next_f 初始复制 f 表示不选。这种写法直接保证同一组至多贡献一件，比混用循环顺序更容易检查。若总物品数为 M，时间 $O(MV)$，空间 $O(V)$。

完整参考实现 18_group.cpp

```cpp
#include <cstdio>
#include <algorithm>
using namespace std;
long long f[2005], next_f[2005], value[25];
int cost[25];
int main() {
    int groups, capacity;
    scanf("%d%d", &groups, &capacity);
    for (int g = 0; g < groups; g++) {
        int k;
        scanf("%d", &k);
        for (int i = 0; i < k; i++) scanf("%d%lld", &cost[i], &value[i]);
        for (int v = 0; v <= capacity; v++) {
            next_f[v] = f[v];
            for (int i = 0; i < k; i++)
                if (v >= cost[i])
                    next_f[v] = max(next_f[v], f[v - cost[i]] + value[i]);
        }
        for (int v = 0; v <= capacity; v++) f[v] = next_f[v];
    }
    printf("%lld\n", f[capacity]);
    return 0;
}
```

9.7 恰好装满时如何处理不可达

将所有状态设为负无穷，仅 f[0]=0。在转移前检查前驱可达，再进行加法。代码可使用约 -4×10^{18} 的哨兵，但必须证明合法值不会接近它；也可用额外布尔数组表示可达。不能把不可达初始化为 0，否则相当于凭空允许每个体积都有一个合法空方案。

9.8 练习与解答

练习 1。只有体积 2 价值 3 的一件物品，容量 3 且要求恰好装满，答案是什么？**解答**。不可达；若要求不超过容量，答案为 3。两种问法的初始化和答案解释不同。

练习 2。面额 2、3 凑 0 的组合数是多少？**解答**。1，即不取任何硬币。这也是 f[0]=1 的组合含义。

练习 3。只有一组，包含体积 2 价值 3、体积 3 价值 5，容量 5。答案是多少？**解答**。5，不能同组选两件得到 8。使用旧数组转移可防止这个错误。

练习 4。若完全背包出现体积 0 且价值正的物品，答案还有限吗？**解答**。没有有限上界。本章题目明确要求体积为正，这是算法与问题有意义的重要前提。

第三部分　第三题：字符串解析与综合模拟

本篇围绕长题意的实现展开：先解析输入，再组织状态，最后执行规则并检查边界。遇到陌生背景时，先列出输入格式、状态定义、修改规则和查询规则，再写程序。

10　字符串处理与解析基础

10.1　常用操作与区间含义

string 的下标从 0 开始。`s.substr(pos,len)` 的第二个参数是长度；`s.find(t)` 返回首次匹配位置，找不到时返回 `string::npos`。不要把 npos 随意存进 int 后再与某个猜测值比较。字符串比较是字典序，例如 `"10" < "2"`，与整数大小不同。

扫描字符串时最好约定："指针 p 总是指向下一个尚未处理的字符"。每个分支要么推进 p，要么返回。若某个分支既不推进也不退出，就容易死循环。处理完整段落时，通常比反复在字符串中间 erase 更清楚、也更高效。

10.2　例题带引号的单行分词

题目。一行文本由普通字符、空格和英文双引号组成。引号外的连续空格分隔单词；引号内的空格属于单词；双引号本身不输出。相邻的普通片段和引号片段属于同一个单词。空引号表示一个空单词。本题不支持转义字符。若引号不匹配，输出 ERROR。行长不超过 10^5。

输入。一整行文本，可为空。**输出**。合法时先输出单词数，随后每个单词单独一行，用方括号包住，便于看见空单词和空格。

```
输入
add "hello world" "" x
输出
4
[add]
[hello world]
[]
[x]
```

状态设计。quoted 表示当前是否在引号内；word[0..len-1] 保存当前单词；active 表示当前单词已经开始。active 不能由 len 是否为 0 代替，否则空引号会丢失。遇到引号切换 quoted；遇到引号外空格时结束当前单词；其余字符追加到 word 并增加 len。扫描结束后检查引号是否闭合，再处理最后一个单词。

正确性。每个字符恰好进入一种分支。引号仅改变状态，分隔空格仅结束一个已开始的单词，因此保留的字符、单词边界和空单词均符合规则。时间 $O(L)$，输出存储 $O(L)$。

完整参考实现 04_tokens.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <string>
#include <vector>
using namespace std;
const int N = 100005;
char s[N], word[N];
int main() {
    if (fgets(s, N, stdin) == NULL) s[0] = '\0';
    s[strcspn(s, "\r\n")] = '\0';
    vector<string> words;
    int len = 0, quoted = 0, active = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c == '"') { quoted = !quoted; active = 1; }
        else if (c == ' ' && !quoted) {
            if (active) {
                word[len] = '\0'; words.push_back(word);
                len = 0; active = 0;
            }
        } else { word[len++] = c; active = 1; }
    }
    if (quoted) { puts("ERROR"); return 0; }
    if (active) { word[len] = '\0'; words.push_back(word); }
    printf("%d\n", (int)words.size());
    for (int i = 0; i < (int)words.size(); i++)
        printf("[%s]\n", words[i].c_str());
    return 0;
}
```

10.3 C 字符数组与 string 的区别

C 字符串以终止字符 '\0' 结尾，数组需要额外空间存放它。strlen 统计终止符前的字符数。定义后的字符数组不能用赋值表达式整体赋值；string 可以直接赋值。strcmp 返回值的符号表示字典序关系，不保证只返回 1 或-1。字符分类函数如 isdigit 接收字符时，可先转为 unsigned char，避免负 char 值带来的未定义行为。

10.4 解析与业务处理分开

例如命令 GRANT alice file1，解析层先产生三个字符串，再由执行层决定如何修改权限。这样出现错误时，可以先打印解析结果，判断错在读入还是状态更新。第 11 章会进一步用函数处理路径和表达式。

10.5 练习与解答

练习 1。输入 a"b c"d，分词结果是什么？**解答**。只有一个单词 ab cd。相邻片段没有引号外空格，不应拆开。

练习 2。输入只有三个空格，输出什么？**解答**。单词数为 0。输入 "" 时则是一个空单词，输出 1 和 []。

练习 3。若题目新增反斜杠转义，当前程序能直接使用吗？**解答**。不能。需要新增 escape 状态：读取反斜杠后，下一个字符按普通字符处理，不切换引号状态。还要定义行末孤立反斜杠是否非法。

规则改变后必须重新列出状态转移。

11 第三题综合模拟专题

11.1 长题意的实现流程

先写一张规则表：输入是什么、状态存在哪里、操作改变什么、查询输出什么、异常如何处理。再按职责拆成解析层、状态层和执行层。只有确认每层的输入输出后，再连成完整程序。下面五个案例规则完整，分别训练路径、映射状态、递归解析、树形结构和依赖展开，可以脱离外部题面学习。

11.2 例题路径规范化

题目。给定固定的当前目录 cwd 和 q 条路径。以斜杠开头的是绝对路径，否则相对于 cwd。连续斜杠视为一个；点号目录表示当前目录；两个点号表示父目录，在根目录继续上移仍为根。普通目录名只含字母、数字、下划线。路径不含空格且非空。

输出规范绝对路径：以一个斜杠开头，不含点号、两个点号或重复斜杠；除根目录外不以斜杠结尾。**每条查询独立使用原来的 cwd，不改变当前目录**。路径长度不超过 10^4，全部输入路径总长度不超过 2×10^5，$1 \le q \le 10^4$。第一行 cwd，第二行 q，随后 q 行各一条路径。cwd 保证为绝对路径，也允许特殊片段。

```
输入
/a/b
5
../c
/x//./y/../
../../../
.
tmp/../../z
输出
/a/c
/x
/
/a/b
/a/z
```

解答。用 vector 保存从根向下的目录名。普通片段入栈，两个点号在非空时弹栈，空片段和点号忽略。绝对路径从空栈开始，相对路径从 cwd 的副本开始。

最后一条查询从 $[a, b]$ 出发，读 tmp 得到 $[a, b, tmp]$，两次父目录操作后为 $[a]$，加入 z 后得到 $[a, z]$。不变量是：栈中的目录始终表示已经处理的路径前缀的规范形式。

设 cwd 长度为 B，第 i 个查询长度为 L_i。复制基目录的总成本最坏为 $O(qB)$，加上扫描与输出为 $O(qB + \sum L_i)$。不能漏算复制和输出中重复的 cwd 长度。

完整参考实现 19_path.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <string>
#include <vector>
using namespace std;
const int N = 10005;
char cwd[N], path[N];
void append_parts(const char s[], vector<string> &parts) {
    char item[N];
    int i = 0;
    while (s[i] != '\0') {
        while (s[i] == '/') i++;
        int len = 0;
        while (s[i] != '\0' && s[i] != '/') item[len++] = s[i++];
        item[len] = '\0';
        if (len == 0 || strcmp(item, ".") == 0) continue;
        if (strcmp(item, "..") == 0) {
            if (!parts.empty()) parts.pop_back();
        } else parts.push_back(item);
    }
}
int main() {
    int q;
    scanf("%10004s%d", cwd, &q);
    vector<string> base;
    append_parts(cwd, base);
    while (q--) {
        scanf("%10004s", path);
        vector<string> result;
        if (path[0] != '/') result = base;
        append_parts(path, result);
        if (result.empty()) printf("/");
        for (int i = 0; i < (int)result.size(); i++)
            printf("/%s", result[i].c_str());
        printf("\n");
    }
    return 0;
}
```

调试。先测根目录与普通绝对路径，再测相对路径和多次上移，最后检查是否错误地修改了共享 cwd。若后续查询受前面查询影响，优先检查是否复用了同一可变目录数组。

11.3 例题授权和撤销的状态管理

题目。执行 q 条命令：`GRANT user resource` 授予权限；`REVOKE user resource` 撤销；`CHECK user resource` 查询并输出 YES 或 NO。重复授予不叠加；撤销不存在的权限不报错；未出现用户默认无权限。操作依次执行，只有 CHECK 输出。$1 \le q \le 2 \times 10^5$，名称为 1 到 20 个字母或数字。本题不含角色继承、通配符或优先级。

输入

```
8
CHECK alice file1
GRANT alice file1
GRANT alice file1
CHECK alice file1
REVOKE alice file1
CHECK alice file1
REVOKE bob file2
CHECK bob file2
```

输出

```
NO
YES
NO
NO
```

解答。外层 map 按用户名定位，内层 set 保存资源。set 处理重复授权。查询和撤销用 find，避免无意创建空记录。所有操作依次执行，因此 CHECK 反映此前的状态。

设用户数 U，每位用户最多持有 R 项权限，单次比较次数为 $O(\log U + \log R)$，还需考虑字符串比较长度。总存储不超过授予过的记录规模。权限是用户与资源的二元关系，不能只判断用户是否出现过；撤销 file1 不能影响 file2。

完整参考实现 20_permissions.cpp

```cpp
#include <cstdio>
#include <cstring>
#include <map>
#include <set>
#include <string>
using namespace std;
map<string, set<string> > rights;
int main() {
    int q;
    scanf("%d", &q);
    while (q--) {
        char op[10], user[21], resource[21];
        scanf("%9s%20s%20s", op, user, resource);
        if (strcmp(op, "GRANT") == 0) rights[user].insert(resource);
        else {
            map<string, set<string> >::iterator it = rights.find(user);
            if (strcmp(op, "REVOKE") == 0) {
                if (it != rights.end()) it->second.erase(resource);
            } else {
                int allowed = it != rights.end() && it->second.count(resource);
                puts(allowed ? "YES" : "NO");
            }
        }
    }
    return 0;
}
```

11.4 例题带括号的整数表达式

题目。输入一行表达式，支持非负整数字面量、加、减、乘、括号和一元正负号。乘法优先于加减，同级二元运算从左到右；一元符号优先于乘法；只忽略普通空格。合法时输出结果，语法错误输出 ERROR。不支持除法、变量和隐式乘法，例如 `2(3+4)` 非法。

行长不超过 1000，括号与连续一元符号嵌套不超过 100 层，字面量不超过 10^6；保证求值过程中涉及的数值运算的中间结果绝对值不超过 10^{12}，本题只讨论语法错误，不要求处理溢出。

输入

```
2 * (3 + 4) - -5
```

输出

```
19
```

另一组输入

```
1 + (2 * 3
```

另一组输出

```
ERROR
```

按语法拆函数。

```
expression = term { ('+' | '-') term }
term       = factor { '*' factor }
factor     = number | '(' expression ')' | '+' factor | '-' factor
```

expression 负责加减，term 负责乘法，factor 负责括号、数字与一元符号。共享指针 p 指向下一个未处理字符，每个函数返回其负责部分的值。term 先完整处理乘法，从而保证优先级；循环保证减法左结合，例如 $8-3-2=3$。

解析前面一个合法表达式不代表整行合法。例如 `1 2`，解析 1 后还剩 2，必须输出 ERROR。最后检查是否读到行尾。时间 $O(L)$，递归空间与嵌套深度有关。

完整参考实现 21_expression.cpp

```cpp
#include <cstdio>
#include <cstring>
char s[1005];
int p = 0, valid = 1;
void skip_space() { while (s[p] == ' ') p++; }
int take(char c) {
    skip_space();
    if (s[p] == c) { p++; return 1; }
    return 0;
}
long long expression();
long long factor() {
    if (take('+')) return factor();
    if (take('-')) return -factor();
    if (take('(')) {
        long long x = expression();
        if (!take(')')) valid = 0;
        return x;
    }
    skip_space();
    if (s[p] < '0' || s[p] > '9') { valid = 0; return 0; }
    long long x = 0;
    while (s[p] >= '0' && s[p] <= '9') x = x * 10 + s[p++] - '0';
    return x;
}
long long term() {
    long long x = factor();
    while (valid && take('*')) x *= factor();
    return x;
}
long long expression() {
    long long x = term();
    while (valid) {
        if (take('+')) x += term();
        else if (take('-')) x -= term();
        else break;
    }
    return x;
}
int main() {
    if (fgets(s, sizeof(s), stdin) == NULL) s[0] = '\0';
    s[strcspn(s, "\r\n")] = '\0';
    long long answer = expression();
    skip_space();
    if (!valid || s[p] != '\0') puts("ERROR");
    else printf("%lld\n", answer);
    return 0;
}
```

调试。依次验证单数字、一元负号、连续加减、乘法优先级、嵌套括号，再测空串、缺右括号、多右括号和多余字符。查看每个函数消费到哪里，比只看最终错误数值更有帮助。

11.5 例题树形文档的后代选择查询

题目。给出按先序排列的一片有根森林，共 n 个节点。每行记录深度、标签和标识。根深度 0，孩子比父亲深 1；输入结构合法，每次深度最多增加 1。标识为减号表示无标识，其余标识互不相同；标签和标识只含字母数字，长度 1 到 20。

查询由空格分隔的选择符组成。普通选择符匹配标签，以井号开头的选择符匹配标识。`div p` 表

示匹配 p 节点，且它的某个**严格祖先**是 div；`div div p` 需要两个不同的 div 祖先按顺序出现，允许隔着其他节点。节点不能把自己当祖先。

$1 \le n \le 500$，$1 \le q \le 50$，最大深度 100，每条查询 1 到 10 个选择符，查询行总长度不超过 1000。第一行 n、q，随后 n 行节点，最后 q 行查询。每次输出匹配数和按输入顺序排列的节点行号，编号从 1 开始。

输入

```
7 5
0 html -
1 body -
2 div main
3 p one
2 div side
3 p two
2 p three
p
div p
#main p
div div p
#side
```

输出

```
3 4 6 7
2 4 6
1 4
0
1 5
```

建树。用祖先栈。读取深度 d 的节点时，把栈截断到长度 d；栈顶就是父亲，空栈则为根，然后压入自己。读取同深度兄弟时，旧兄弟会被弹出。

匹配。从查询末尾开始，先检查候选节点本身，再沿父链逆序匹配剩余选择符。找到一个就继续寻找更高祖先。选择最近的匹配祖先不会损失可行解，因为它给剩余前缀保留最多的上方祖先。找齐全部选择符则接受。

候选检查最多走树高 H 条父边并处理 K 个选择符，整体为 $O(qn(H+K))$，本题规模下足够。节点 7 虽是 p，但没有 div 祖先；若程序把它算入 `div p`，通常是把 “之前出现的节点” 误认为 “当前祖先”。

完整参考实现 22_selector.cpp

```cpp
#include <cstdio>
#include <cstring>
struct Node { char tag[21], id[21]; int parent; };
Node a[505];
int ancestors[505], answer[505];
char line[1005], tokens[10][22];
int matches(int u, const char token[]) {
    if (token[0] == '#')
        return strcmp(a[u].id, "-") != 0 && strcmp(a[u].id, token + 1) == 0;
    return strcmp(a[u].tag, token) == 0;
}
int main() {
    int n, q, top = 0;
    scanf("%d%d", &n, &q);
    for (int i = 0; i < n; i++) {
        int depth;
        scanf("%d%20s%20s", &depth, a[i].tag, a[i].id);
        top = depth;
        a[i].parent = top == 0 ? -1 : ancestors[top - 1];
        ancestors[top++] = i;
    }
    int ch;
    while ((ch = getchar()) != '\n' && ch != EOF) {}
    while (q--) {
        if (fgets(line, sizeof(line), stdin) == NULL) break;
        int count = 0;
        char *s = strtok(line, " \t\r\n");
        while (s != NULL) {
            strcpy(tokens[count++], s);
            s = strtok(NULL, " \t\r\n");
        }
        int total = 0;
        for (int u = 0; u < n; u++) {
            int k = count - 1;
            if (k < 0 || !matches(u, tokens[k])) continue;
            k--;
            int v = a[u].parent;
            while (v != -1 && k >= 0) {
                if (matches(v, tokens[k])) k--;
                v = a[v].parent;
            }
            if (k < 0) answer[total++] = u + 1;
        }
        printf("%d", total);
        for (int i = 0; i < total; i++) printf(" %d", answer[i]);
        printf("\n");
    }
    return 0;
}
```

11.6 例题模板展开与循环依赖

题目。有 n 个命名模板，每个由若干以空白分隔的词组成。以 at 符号开头的词引用另一个模板，其余词直接输出。定义全部读完后，展开指定起始模板，以单个空格连接所有普通词。从起始模板可达的引用若出现未定义名称或环，输出 ERROR；不可达模板的错误不影响本次输出。

模板名互不重复，只含字母数字，长度不超过 20；引用符号后必须有名称。允许空模板。$1 \le n \le 100$，每个定义最多 100 个词，每词长度最多 20。保证无环情况下每个模板完整展开不超过 2000 个词。第一行 n，随后每个定义包含名称、词数 k 及 k 个词，最后给出起始名称。

输入

```
3
title 1 world
line 3 hello @title !
all 2 @line @line
all
```

输出

```
hello world ! hello world !
```

另一组输入

```
2
a 1 @b
b 1 @a
a
```

另一组输出

```
ERROR
```

三色状态。0 表示未算，1 表示正在递归，2 表示已经完成并缓存。访问状态 1 说明当前依赖链回到了自己；访问状态 2 直接复用。第二次引用同一模板不必然成环，关键看第一次展开是否结束。

解答。先读入全部定义以允许前向引用。按词序展开，普通词加入结果，引用词接上被引用模板的结果。记忆化避免重算，但每次拼接仍复制相应词。时间取决于查找及所有实际复制的词数；本题每个模板最多缓存 2000 词，总缓存约不超过 100×2000 词，不能忽略输出规模而声称总是 $O(n)$。

完整参考实现 23_templates.cpp

```cpp
#include <cstdio>
#include <map>
#include <string>
#include <vector>
using namespace std;
map<string, int> number;
vector<string> definition[105], memo[105];
int state[105];
int expand(int u) {
    if (state[u] == 1) return 0;
    if (state[u] == 2) return 1;
    state[u] = 1;
    for (int i = 0; i < (int)definition[u].size(); i++) {
        string token = definition[u][i];
        if (token[0] != '@') memo[u].push_back(token);
        else {
            map<string, int>::iterator it = number.find(token.substr(1));
            if (it == number.end() || !expand(it->second)) return 0;
            int v = it->second;
            for (int j = 0; j < (int)memo[v].size(); j++)
                memo[u].push_back(memo[v][j]);
        }
    }
    state[u] = 2;
    return 1;
}
int main() {
    int n;
    scanf("%d", &n);
    char name[21], token[21];
    for (int i = 0; i < n; i++) {
        int k;
        scanf("%20s%d", name, &k); number[name] = i;
        for (int j = 0; j < k; j++) {
            scanf("%20s", token); definition[i].push_back(token);
        }
    }
    scanf("%20s", name);
    map<string, int>::iterator it = number.find(name);
    if (it == number.end() || !expand(it->second)) puts("ERROR");
    else {
        int u = it->second;
        for (int i = 0; i < (int)memo[u].size(); i++)
            printf("%s%s", i == 0 ? "" : " ", memo[u][i].c_str());
        printf("\n");
    }
    return 0;
}
```

11.7 练习与解答

练习 1。cwd 为根，路径为 `../../a`，结果是什么？**解答**。`/a`，超出根的上移被忽略。

练习 2。alice 有 A 和 B，撤销 A 后查询 B 输出什么？**解答**。YES，资源权限分别维护。

练习 3。`-2*-3+4` 的值是多少？**解答**。10。一元负号属于 factor，乘法先得 6，再加 4。

练习 4。一个 div 节点能否独自匹配 `div div`？**解答**。不能，两个选择符必须匹配不同节点，前一个位于严格祖先链。

练习 5。A 引用 B 两次，B 只含 x，是否有环？**解答**。没有。第一次结束后 B 为状态 2，第二

次复用，输出 x x。

12 综合训练与复盘

12.1 练习方式

本章三道题均为教学训练题，不对应某次认证的原版试卷。建议先不看解答，给自己安排 30、50、90 分钟的独立尝试，再用解答检查建模过程。用时只是训练建议，不是官方考试分配。

12.2 例题训练 A 差值数对

给 n 个整数和非负整数 c，求满足 $i < j$ 且 $|a_i - a_j| = c$ 的位置对数量。$1 \le n \le 2 \times 10^5$，$|a_i| \le 10^9$，$0 \le c \le 2 \times 10^9$。第一行 n、c，第二行数组，输出一个整数。

输入

```
5 2
1 3 1 3 3
```

输出

```
6
```

解答。先统计每个值的频次。c>0 时，只从较小值 x 寻找 x+c，不会重复统计同一无序数对；贡献是两组频次乘积。c=0 时，频次 k 的组中任选两个位置，贡献 $k(k-1)/2$。答案最多为 $n(n-1)/2$，需要 long long。时间 $O(n \log n)$，空间 $O(n)$。

完整参考实现 24_pairs.cpp

```cpp
#include <cstdio>
#include <map>
using namespace std;
int main() {
    int n; long long c;
    scanf("%d%lld", &n, &c);
    map<long long, long long> count;
    for (int i = 0; i < n; i++) {
        long long x; scanf("%lld", &x); count[x]++;
    }
    long long answer = 0;
    map<long long, long long>::iterator it;
    for (it = count.begin(); it != count.end(); ++it) {
        long long x = it->first, k = it->second;
        if (c == 0) answer += k * (k - 1) / 2;
        else {
            map<long long, long long>::iterator p = count.find(x + c);
            if (p != count.end()) answer += k * p->second;
        }
    }
    printf("%lld\n", answer);
    return 0;
}
```

检查用例。数组 [7, 7, 7]、c=0 应输出 3；单元素时应输出 0。若输出 9，说明把相同位置配对或对称重复都算了进去。

12.3 例题训练 B 矩形增量的最大值

初始有 $n \times m$ 全 0 矩阵，执行 q 次操作，每次给闭矩形左上 (x_1, y_1) 到右下 (x_2, y_2) 加 v。最终输出最大元素值，以及等于最大值的格子数。$1 \le n, m \le 1000$，$0 \le q \le 2 \times 10^5$，$|v| \le 10^6$，坐标合法。第一行 n、m、q，随后每行五个数。

输入

```
3 3 2
1 1 2 2 2
2 2 3 3 3
```

输出

```
5 1
```

解答。把每次矩形更新记录在二维差分四角，最后恢复每格真实值，同时维护最大值和次数。只统计 1 到 n 行、1 到 m 列，不能把辅助边界算进去。允许负更新，最大值要从负无穷开始，不能默认 0。时间 $O(nm + q)$，空间 $O(nm)$。

完整参考实现 25_rectangle.cpp

```cpp
#include <cstdio>
#include <climits>
const int N = 1005;
long long d[N][N];
int main() {
    int n, m, q;
    scanf("%d%d%d", &n, &m, &q);
    while (q--) {
        int x1, y1, x2, y2; long long v;
        scanf("%d%d%d%d%lld", &x1, &y1, &x2, &y2, &v);
        d[x1][y1] += v; d[x2 + 1][y1] -= v;
        d[x1][y2 + 1] -= v; d[x2 + 1][y2 + 1] += v;
    }
    long long best = LLONG_MIN, count = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            d[i][j] += d[i - 1][j] + d[i][j - 1] - d[i - 1][j - 1];
            if (d[i][j] > best) { best = d[i][j]; count = 1; }
            else if (d[i][j] == best) count++;
        }
    }
    printf("%lld %lld\n", best, count);
    return 0;
}
```

检查用例。q=0 时输出 0 n*m，第二项应计算成实际格子数；若整个矩阵一次减 5，则最大值为-5 而不是 0。

12.4 例题训练 C 非抢占任务调度

有一台处理器和 n 个任务，每个任务给定到达时刻、优先级和执行时间，输入顺序就是任务编号。处理器空闲时，在所有已经到达但尚未执行的任务中选择优先级最高者；并列时选到达更早者，再并列选编号更小者。一旦开始执行就**不能被抢占**。

一个任务完成时，必须先把恰好在该时刻到达的任务加入候选，再选择下一项。若没有已到达任务，时钟直接跳到下一次到达。输入不保证按到达时刻有序。$1 \le n \le 2 \times 10^5$，到达时刻和优先

级在 0 到 10^9 之间，执行时间在 1 到 10^9 之间。

第一行 n，随后每行依次给到达时刻、优先级、执行时间。按输入编号输出每个任务的开始和结束时刻，最后一行输出最大等待时间，等待时间为开始减到达。

输入

```
4
0 1 5
1 1 2
2 3 1
5 2 2
```

输出

```
0 5
8 10
5 6
6 8
7
```

推演。任务 1 在 0 到 5 执行，中途到达的高优先级任务不能打断它。时刻 5，先加入任务 4；待选任务 2、3、4 中任务 3 优先级最高，所以执行顺序为 1、3、4、2。任务 2 等到 8 开始，等待 7。

状态划分。排序数组保存还未到达的任务，指针 i 指向下一项；优先队列保存已经到达的任务；now 保存处理器可再次分配工作的时刻。每轮先补全所有到达时间不大于 now 的任务，再选堆顶。

正确性。选择前，堆恰好包含全部可选任务；比较器严格实现三级优先顺序。执行结束后更新 now，不处理中途抢占，符合规则。每个任务入堆出堆各一次，排序与堆操作总时间 $O(n \log n)$，空间 $O(n)$。结束时刻可能达到 10^{14} 量级，应使用 long long。

完整参考实现 26_scheduler.cpp

```cpp
#include <cstdio>
#include <algorithm>
#include <queue>
#include <vector>
using namespace std;
const int N = 200005;
struct Task { long long arrival, priority, duration; int id; };
Task tasks[N];
long long start_time[N], finish_time[N];
bool by_arrival(Task a, Task b) {
    if (a.arrival != b.arrival) return a.arrival < b.arrival;
    return a.id < b.id;
}
bool lower_priority(Task a, Task b) {
    if (a.priority != b.priority) return a.priority < b.priority;
    if (a.arrival != b.arrival) return a.arrival > b.arrival;
    return a.id > b.id;
}
int main() {
    int n;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        scanf("%lld%lld%lld", &tasks[i].arrival,
              &tasks[i].priority, &tasks[i].duration);
        tasks[i].id = i;
    }
    sort(tasks, tasks + n, by_arrival);
    typedef bool (*Compare)(Task, Task);
    priority_queue<Task, vector<Task>, Compare> ready(lower_priority);
    long long now = 0, max_wait = 0;
    int i = 0;
    while (i < n || !ready.empty()) {
        if (ready.empty()) now = max(now, tasks[i].arrival);
        while (i < n && tasks[i].arrival <= now) ready.push(tasks[i++]);
        Task t = ready.top(); ready.pop();
        start_time[t.id] = now; finish_time[t.id] = now + t.duration;
        max_wait = max(max_wait, now - t.arrival);
        now += t.duration;
    }
    for (int j = 0; j < n; j++)
        printf("%lld %lld\n", start_time[j], finish_time[j]);
    printf("%lld\n", max_wait);
    return 0;
}
```

12.5 如何复盘一次错误

先保存最小失败输入，写出期望输出，再归类原因。输入解析错误要检查分词和换行；状态错误要明确变量对应时刻；算法错误要重新验证单调性、转移或贪心理由；复杂度错误要估计最坏规模。修复后还要补一个能够稳定复现原错误的测试。

例如调度程序把新任务加入堆的条件写成“到达时刻小于 now”，就会漏掉恰好在任务结束时到达的任务。用“任务 1 在 0 开始、执行 2；任务 2 在 2 到达”即可定位边界。改成小于等于后，再检查已有等待任务与同时到达任务的优先顺序。

12.6 提交前检查表

1. 输出内容、顺序和格式是否与题意一致，有没有调试输出残留。
2. 最小输入、全相等、全负数、空结果、不可达等合法边界是否处理。
3. 数值和数组范围是否足够，r+1、i-1 等辅助位置是否分配。
4. 每个循环能否终止；每个递归状态是否完整；是否可能爆栈。
5. 每个容器访问前是否检查为空或 end；比较器是否严格。
6. 程序复杂度是否覆盖最大数据，而不仅是样例。

A 提高专题与模型扩展

本附录承接原资料中的提高内容。建议在完成正文例题后选择学习。每次变换模型时，先检查状态语义是否改变，不要仅凭代码形式相似就替换循环。

A.1 多重背包与二进制拆分

第 i 种物品最多有 s_i 件。朴素转移枚举件数 k：

$$f[i][v]=\max_{0\le k\le s_i,\ kc_i\le v}\{f[i-1][v-kc_i]+kw_i\}.$$

当数量较大时，把 s_i 件拆成若干只能选一次的物品包。例如 13 拆成 1、2、4、6，每包体积和价值乘相应件数。选择这些包可以表示 0 到 13 的任意件数，转化为 01 背包。拆包数量为 $O(\log(s_i+1))$，总时间为 $O(V\sum_i\log(s_i+1))$。

```
int remaining = count;
for (int batch = 1; remaining > 0; batch *= 2) {
    int take = min(batch, remaining);
    remaining -= take;
    long long cc = 1LL * take * cost;
    long long ww = 1LL * take * value;
    if (cc <= capacity)
        for (int v = capacity; v >= cc; --v)
            f[v] = max(f[v], f[v-(int)cc] + ww);
}
```

此片段假设 count 不超过 10^6、体积为正，避免 batch 翻倍溢出。若采用更大数量范围，应使用更宽类型并检查翻倍边界。

为什么能覆盖。若已有包可以组成 0 到 t，再加入大小不超过 t+1 的新包，就可以连续覆盖到 t 加新包大小。按 1、2、4 增长并保留余数满足这个条件。

计数陷阱。这个拆分通常用于最优值或可达性，不能直接拿来统计原问题的方案数。例如拆成 1、2、4、6 后，6 可由 6 或 2+4 表示，形成不同拆包选择却代表同样件数。计数背包应另行设计不重不漏的转移。

A.2 混合背包与多维费用

混合背包对每种物品分别判断：限一次用逆序，无限次用正序，有限次可枚举或拆包；组内互斥则按整组处理，不能把同组物品当普通独立物品。不同模型混在一起时，应确保每轮转移的“一

个决策单位”仍然清晰。

若每件物品同时占体积 c 和重量 d，容量限制为 V 和 W，定义 $f[v][w]$ 为两种资源限制下的最大价值。对于 01 物品，两个容量维度都逆序：

```
for (int v = V; v >= cost; --v)
    for (int w = W; w >= weight; --w)
        f[v][w] = max(f[v][w], f[v-cost][w-weight]+value);
```

时间 $O(nVW)$，空间 $O(VW)$。增加状态维度会迅速增加计算量。应选择取值范围较小的量作维度，必要时考虑按价值记录最小成本，但也要重新定义答案和不可达值。

A.3 方案恢复

若要求输出选了哪些物品，仅得到最大值还不够。对于二维 01 背包，可在每个状态记录是否选择第 i 件，或保留整张 f 表，从 (n, V) 逆推：若最优值来自不选则 i 减 1；若来自选择，则记录 i 并把容量减去体积。

一维压缩后直接保存“当前容量的父指针”容易被后续物品覆盖，造成恢复结果重复选同一物品。为了初学者的可靠实现，先使用二维记录恢复，再考虑持久化等高级优化。并列最优解允许任选时，固定一种规则即可；要求字典序最小时必须把并列规则也纳入设计。

A.4 计数背包的撤销

设 F[s] 是所有物品组成体积 s 的 01 选择方案数，删除一个体积为 w 的**指定物品**后，G[s] 是剩余方案数。按是否选该物品分类：

$$F[s] = G[s] + G[s-w], \qquad G[s] = F[s] - G[s-w].$$

负下标项为 0。因 G[s] 依赖更小的 G，按 s 从小到大计算。若只输出末位数字，应在模 10 下减法并把负数归一化：

```
for (int s = 0; s <= capacity; s++) g[s] = all[s];
for (int s = w; s <= capacity; ++s)
    g[s] = (all[s] - g[s-w] + 10) % 10;
```

例如两个体积都为 1 的不同物品，完整方案数为 $F[0..2] = [1, 2, 1]$；删去其中一件，得到 $G = [1, 1, 0]$。物品即使体积相同也按身份区分。该线性计数关系可撤销，但最大值背包的 max 会丢失候选信息，不能简单相减恢复。

A.5 LIS 优化与排列 LCS

定义 tail[len] 为所有长度 len 的严格上升子序列中最小的末尾值。这个数组严格递增。处理 x 时，用 lower_bound 找到第一个不小于 x 的位置并替换；若不存在就把 x 接到末尾。替换更小的末尾不会改变已获得的长度，却给后续元素留下更多延伸可能。

```
int length = 0; // tail[] and a[] have capacity at least n.
for (int i = 0; i < n; i++) {
    int pos = lower_bound(tail, tail + length, a[i]) - tail;
    tail[pos] = a[i];
    if (pos == length) length++;
}
printf("%d\n", length);
```

时间 $O(n \log n)$。tail 的内容不一定是一条真实的 LIS；若要求恢复路径，还需要记录位置和前驱。不下降子序列对应 upper_bound，而不是 lower_bound。

若两个序列都是 1 到 n 的排列，把第一序列每个值的位置记为 pos，再将第二序列映射成位置序列。共同子序列需要在第一序列位置上递增，因此转成 LIS。这个转换利用了元素唯一的性质，不能直接用于有任意重复值的一般 LCS。

A.6 实数二分与三分的前提

实数二分可固定迭代次数，避免直接比较浮点相等。例如连续函数在区间两端异号，可以保留仍有变号的半区间来寻找一个根。端点恰好为根时先处理；两个端点同号不代表没有根，也不能保证可以用这一规则寻找全部根。三次方程若有多个根，应先按题目给定性质隔离各根。

三分用于已知单峰或单谷的目标函数。对单峰求最大值，比较两个三等分点，舍弃不可能包含峰值的一侧；若函数存在多个峰，这个依据就不成立。分数规划、爬山和模拟退火还各有额外假设，不能用“都在搜索答案”把它们当作二分的替代模板。基础前三题训练应先把可证明的单调模型掌握牢固。

A.7 可删堆的延迟删除

优先队列通常只支持删除堆顶。若外部能保证删除请求对应一个当前存在的元素，可维护两个同类型堆：A 保存所有插入值，B 保存待删除值。每次取有效堆顶前，若两堆顶相等就同时弹出，直到不同或有堆为空。

重复值应按次数插入删除堆，不是用布尔标记；还要维护逻辑元素数，以区分真实为空与仍有待清理的节点。延迟删除让每个入堆元素最多被清理一次，但实现前应先判断 multiset 是否已经足够简单可靠。

A.8 字符串匹配的线性预处理

直接在每个起点比较模式串的复杂度最坏为 $O(nm)$，在小规模下完全可用。规模较大时，可用 KMP 的前缀函数维护“已匹配前缀还能退到多长”。设 pi[i] 为模式串前缀 0 到 i 的最长相等真前后缀长度：

```cpp
void prefix_function(const char s[], int pi[]) {
    int n = strlen(s);
    if (n == 0) return;
    pi[0] = 0;
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }
}
```

对非空模式 p 扫描文本时，用同样的回退规则更新匹配长度 j；当 j 等于模式长度，记录一次匹配并回退到 pi[j-1]，从而允许重叠匹配。由于指针前进与回退的总量受线性界约束，预处理和匹配合计 $O(n+m)$。空模式的匹配规则应另行规定，不能直接访问 p[0]。

A.9 附录练习与解答

练习 1。数量 10 如何拆包？**解答**。1、2、4、3，和为 10，能表示 0 到 10 的每个件数。

练习 2。两个排列 $[3,1,2]$ 与 $[1,3,2]$ 的 LCS 长度是多少？**解答**。第一排列位置为 pos[3]=1、pos[1]=2、pos[2]=3；第二个映射为 $[2,1,3]$，LIS 长度 2，所以 LCS 长度 2。

练习 3。模式 aaa 在文本 aaaa 中出现几次？**解答**。2 次，起点 0 和 1。记录第一次后应保留可重叠的前后缀，而不是把匹配长度清成 0。

B 资料来源与继续练习

本讲义依据以下原始材料重新编排知识体系，补充了自包含教学例题、解答和参考程序。原课件中的重点概念问题已在正文采用修正后的表述，包括 unique 语义、DP 状态信息、背包初始化、BFS 归类和计数背包撤销。

原始文件	页数	本讲义对应内容
历年前三题考点梳理.pdf	5	题型安排、基础模拟、综合模拟选题方向。
练习题与真题.pdf	2	继续训练入口，仓库中另存链接索引。
stl.pptx	42	第 3 章 STL 专章、第 10 章解析与可删堆。
前缀和、差分、二分、双指针.pptx	34	第 4–5 章及搜索答案的选读内容。
搜索、剪枝、暴力.pptx	13	第 7 章与模板依赖展开。
dp 入门.pptx	18	第 8 章、LIS 与 LCS 扩展。
动态规划之背包问题.pptx	53	第 9 章与背包提高专题。

基础题进一步练习可选：排序 P1177、去重 P1059、前缀和 P8218、差分 P2367、二分 P2249、

切木条 P2440、全排列 P1706、马的遍历 P1443、LIS B3637、最大子段和 P1115、01 背包 P1048、完全背包 P1616、分组背包 P1757。这些题可能有与教学题不同的输入、限制和输出要求，应重新阅读原题。

提高题可选：P2678、P1219、P3956、P1776、P1507、P1064、P1439、P4141。其中 P4141 的目标是删除指定物品后的恰好装满方案数末位数字，不是最大价值；P1439 的较大数据需要利用排列性质，普通二维 LCS 不能覆盖全部规模。

接口语义可查阅 C++ 工作草案的sort 条目和unique 条目。平台原题和标准文档是进一步核对的依据，本讲义正文的教学例题与解答已经提供独立学习所需的规则和算法。